

Sisteme Operative

Ligj. 1

- 1) **Sistemi Operativ** është një softuer i cilin menaxhon harduerin e një kompjuteri. Sistemi operativ ofron një bazë për programet aplikative dhe luan rolin e një ndërmjetësuesi në mes të shfrytëzuesit të kompjuterit dhe harduerit të kompjuterit.
- 2) Një sistem kompjuterik mund të ndahet në katër pjesë: hardueri, sistemi operativ, programet aplikative dhe shfrytëzuesi.
 - a. **Hardueri** – njësia qendrore përpunuese (CPU), memoria dhe pajisjet hyrëse/dalëse – ofron resurset bazë kompjuterike për sistemin.
 - b. **Programet aplikative** – si programet për përpunimin e teksteve, për llogaritje, kompajlerët dhe ueb shfletuesit – përcaktojnë mënyrat se si këto resurse përdoren për të zgjidhur probleme të shfrytëzuesit me anë të kompjuterit. Sistemi operativ kontrollon harduerin dhe kordinon përdorimin e tij në mes programeve të ndryshme aplikative për shfrytëzues të ndryshëm.
- 3) Sistemi operativ vepron si menaxhues i këtyre resurseve. Duke u përballë me shumë kërkesa dhe me mundësinë që ato të jenë në konflikt me njëra-tjetrën, sistemi operativ duhet patjetër të vendosë se si të japë (ndajë) resurset për programet dhe shfrytëzuesit e caktuar ashtu që të mund të punojë me sistemin kompjuterik në mënyrë eficiente dhe të drejtë.
- 4) Sistemi operativ është një program për kontroll. Një program kontrolli menaxhon ekzekutimin e programeve të shfrytëzuesit për të parandaluar gabimet dhe përdorimin jo të duhur të kompjuterit. Merret posaçërisht me punën dhe kontrollin e pajisjeve hyrëse/dalëse.
- 5) Qëllimi themelor i sistemeve kompjuterike është ekzekutimi i programeve dhe të bëjnë të lehtë zgjidhjen e problemeve të shfrytëzuesit.
- 6) Sistemi operativ është programi që ekzekutohet gjatë tërë kohës (gjatë kohës kur kompjuteri është duke punuar) në kompjuter – zakonisht i njohur si kerneli (bërthama ose pjesa kryesore). Krahas kernelit, janë edhe dy lloje të softuerëve: programet e sistemit, që janë pjesë e sistemit operativ, por që nuk janë domosdosmërisht pjesë e kernelit, dhe programet aplikative, që përshijnë të gjitha programet që nuk janë pjesë e sistemit operativ.
- 7) Sistemet operative të pajisjeve mobile zakonisht përfshijnë jo vetëm një kernel por edhe middleware – një bashkësi e kornizave (frameworks) për zhvillim të softuerit që ofrojnë shërbime shtesë për zhvilluesit e aplikacioneve.
- 8) Sistemi operativ përshinë kernelin që ekzekutohet gjatë tërë kohës, kornizat middleware që lehtësojnë zhvillimin e aplikacioneve, ofrojnë karakteristika dhe programe të sistemit që ndihmojnë në menaxhimin e sistemit gjatë kohës kur është duke punuar.
- 9) Programi fillues i njohur si programi bootstrap zakonisht është i thjeshtë. Zakonisht ruhet brenda harduerit në firmware. Fillon të gjitha pjesët e sistemit kompjuterik, që nga regjistrat e CPU-së te kontrolluesit e pajisjeve e deri te përmbajtjet e memories. Programi bootstrap duhet patjetër të jetë në gjendje të vendosë sistemin operativ në memorien DRAM dhe të fillojë punën e sistemit. Për të realizuar këtë punë, programi bootstrap duhet të gjejë kernelin e sistemit operativ (në disk SSD ose HDD) dhe të vendosë atë në memorien DRAM.
- 10) Disa nga shërbimet ofrohen jashtë kernelit nga programet e sistemit të cilat vendosen në memorien DRAM gjatë kohës së fillimit të punës të sistemit kompjuterik për t'u bërë programe

të njohura si programe daemon, të cilat ekzekutohen gjatë tërë kohës kur është duke u ekzekutuar kerneli.

- 11) Paraqitja e ndonjë ngjarjeje sinjalizohet përmes ndonjë ndërprerjeje (interrupt). Një lloj tjetër i ndërprerjes është ndërprerja trap (ose një exception), e cila është një ndërprerje e shkaktuar nga softueri nga ndonjë gabim (për shembull, pjesëtimi me zero ose qasje jo valide në memorie) ose ndonjë kërkesë specifike e një programi të shfrytëzuesit që një shërbim i sistemit operativ të kryhet duke ekzekutuar një operacion të veçantë të njohur si system call (thirrje sisteme).
- 12) **Multiprogramimi** e rritë përdorimin e CPU-së, duke i bërë të lumtur edhe shfrytëzuesit, me organizimin e ekzekutimit të programeve ashtu që CPU në çdo kohë ka një program për ekzekutim. Në një sistem me multiprogramim, një program që është duke u ekzekutuar njihet si proces.
- 13) **Multitasking** është një zgjerim i natyrshëm i multiprogramimit. Në sistemet me multitasking, CPU ekzekuton shumë procese duke kaluar nëpër to, por kalimi bëhet shpesh, duke i ofruar shfrytëzuesit një kohë reagimi të shkurtër.
- 14) Ekzistenca në të njëjtën kohë e disa proceseve në memorien DRAM kërkon një mënyrë të menaxhimit të memories. Për më tepër, nëse disa procese janë gati për ekzekutim në të njëjtën kohë, sistemi duhet të përzgjedhë se cili proces do të ekzekutohet në vazhdim. Kjo punë paraqet planifikim të kohës së CPU-së.
- 15) Sistemet me multiprogramim dhe multitasking duhet patjetër të ofrojnë një sistem të fajllave (file system). **Sistemi i fajllave** ndodhet në një memorie me kapacitet të madh; në atë mënyrë, duhet patjetër të ofrohet menaxhim i ruajtjes. Sistemi duhet patjetër të ofrojë edhe mbrojtje të resurseve nga përdorimi jo i duhur. Për të siguruar ekzekutim të rregullt, sistemi duhet patjetër të ofrojë mekanizma për sinkronizim dhe komunikim në mes të proceseve, e të sigurojë që proceset të mos bllokohen në një situatë deadlock, duke pritur përgjithmonë për njëritjetrin.
- 16) Qasja që përdoret në shumë sisteme kompjuterike është që të ofrohet përkrahje harduerike që lejon dallimin në mes të mënyrave të ndryshme të ekzekutimit. Do të kishim nevojë për së paku dy mënyra të ndara të punës: mënyra e shfrytëzuesit (user mode) dhe mënyra e kernelit (e njohur ndryshe edhe si mënyra e mbikëqyrësit (supervisor mode), system mode ose privileged mode). Një bit, i quajtur si biti i mënyrës (mode bit), i shtohet harduerit të kompjuterit për të treguar mënyrën aktuale të punës: kerneli (biti 0) ose shfrytëzuesi (biti 1). Përmes bitit të mënyrës, mund të dallojmë një punë që është duke u kryer për sistemin operativ dhe një punë që është duke u kryer për shfrytëzuesin.
- 17) Puna me dy mënyra të punës ofron mundësinë e mbrojtjes së sistemit operativ nga shfrytëzuesit problematik – dhe shfrytëzuesit problematik nga njëri-tjetri. Kjo mbrojtje arrihet duke i trajtuar disa instruksione të cilat mund të shkaktojnë dëme (nëse keqpërdoren) si instruksione të privileguara. Hardueri lejon instruksionet e privileguara të ekzekutohen në mënyrën e kernelit. Nëse ka ndonjë tentim që një instruksion i privileguar të ekzekutohet në mënyrën e shfrytëzuesit, hardueri nuk e ekzekuton instruksionin, por e trajton si jo të lejueshëm dhe e kthen kontrollin te sistemi operativ.
- 18) Kur një thirrje sisteme ekzekutohet, hardueri zakonisht e trajton si ndërprerje softuerike. Kontrolli bartet përmes vektorit të ndërprerjes të një procedurë në sistemin operativ dhe vendoset mënyra kernel. Procedura (funksioni) e cila merret me thirrjen sisteme është pjesë e sistemit operativ. Kernel e analizon instruksionin që ka shkaktuar ndërprerje për të përcaktuar cila thirrje sisteme është paraqitur; një parametër tregon se çfarë lloji të shërbimit kërkon

programi i shfrytëzuesit. Informatat tjera shtesë të kërkesës mund të barten në regjistra, në stek ose në memorie (duke i vendosur në regjistra adresat e vendeve në memorie). Kerneli i vërteton parametrat se a janë mirë dhe të lejueshëm, ekzekuton kërkesën dhe kthen kontrollin te instruksioni që ndodhet pas instuksionit që ka shkaktuar thirrje sistmore.

- 19) Duhet të sigurohemi që sistemi operativ e mirëmban kontrollin mbi CPU-në. Nuk duhet lejuar që një program i shfrytëzuesit të bllokohet në një unazë të pafundme ose të dështojë në thirrjen e shërbimeve të sistemit dhe të mos kthejë më kontrollin te sistemi operativ. Për të arritur këtë qëllim, mund të përdorim një tajmer.
- 20) Vetë programi nuk është një proces. Një program është një entitet pasiv, sikur përmbajtjet e një fajlli të ruajtur në disk, ndërsa një proces është një entitet aktiv. Një proces me vetëm një thred ka një regjistër PC (program counter) për adresën e instruksonit të radhës për ekzekutim.
- 21) Sistemi operativ është përgjegjës për këto aktivitete në lidhje me menaxhimin e proceseve:
 - a. Krijimi dhe fshirja e proceseve të shfrytëzuesit dhe të sistemit
 - b. Planifikimi për ekzekutim të proceseve dhe thredave në CPU
 - c. Ndalja e përkohshme dhe vazhdimi i proceseve
 - d. Ofrimi i mekanizmave për sinkronizim të proceseve
 - e. Ofrimi i mekanizmave për komunikim në mes të proceseve
- 22) Sistemi operativ është përgjegjës për këto aktivitete në lidhje me menaxhimin e memories:
 - a. Mbajtja e shënimeve se cilat pjesë të memories janë duke u përdorë në momentin e caktuar dhe cili proces është duke i përdorë ato
 - b. Dhënia dhe marrja e hapësirës së memories sipas nevojës
 - c. Marrja e vendimit se cilat procese (ose pjesë të proceseve) dhe të dhëna duhet të vendosen dhe të largohen nga memoria
- 23) Fajlli është një grumbull i informatave të lidhura në mes vete të përcaktura nga krijuesi i fajllit. Zakonisht, fajllat paraqesin programe (në formë të kodit burimor dhe në formë të kodit të kompajluar) dhe të dhëna.
- 24) Sistemi operativ është përgjegjës për këto aktivitete në lidhje me menaxhimin e fajllave:
 - a. Krijimi dhe fshirja e fajllave
 - b. Krijimi dhe fshirja e direktorumeve për organizimin e fajllave
 - c. Përkrahja e komandave elementare për punë me fajlla dhe direktorume
 - d. Pasqyrimi (vendosja) i fajllave në memoriet me kapacitet të madh
 - e. Ruajtja e një kopjeje të fajllave në memorie që i ruajnë të dhënat edhe pas ndërprerjes së furnizimit me elektricitet
- 25) Sistemi operativ është përgjegjës për këto aktivitete në lidhje me menaxhimin e pajisjeve me madhësi të madhe:
 - a. Vendosja dhe largimi në sistem
 - b. Menaxhimi i hapësirës së lirë
 - c. Planifikimi i përdorimit të diskut
 - d. Ndarja e diskut në pjesë
 - e. Mbrojtja
- 26) Meqë keshet kanë madhësi të kufizuar, menaxhimi i keshit është një problem i rëndësishëm i dizajnit. Përzgjedhja me kujdes e madhësisë së keshit dhe algoritmit për zëvendësim mund të rezultojë në rritje të performancës.

- 27) Njëri nga qëllimet e një sistemi operativ është të fshehë detajet e pajisjeve të caktuara harduerike nga shfrytëzuesi. Për shembull, në sistemin operativ UNIX, detajet e pajisjeve të hyrje/daljes fshehen nga pjesa tjetër e sistemit operativ përmes nënsistemit të hyrje/daljes. Nënsistemi i hyrje/daljes përbëhet nga disa pjesë:
- Pjesa për menaxhim të memories që përfshinë baferimin, keshimin dhe spooling
 - Një interfejs i përgjithshëm për drajverin e pajisjes
 - Drajverë për pajisje harduerike të caktuara Vetëm drajveri i pajisjes i din detajet e pajisjes së caktuar të cilën e menaxhon.
- 28) **Mbrojtja** është çdo mekanizëm për kontrollimin e qasjes së proceseve ose të shfrytëzuesve në resurse të përcaktuara nga sistemi kompjuterik. Ky mekanizëm duhet patjetër të ofrojë një mundësi të vendosjes dhe zbatimit të kontroleve. Mbrojtja mund të përmirësojë besueshmërinë duke gjetur gabime të fshehura në interfejset në mes të nënsistemeve. Gjetja e hershme e gabimeve në interfejs (pikat e kontaktit) shpesh mund të parandalojë infektimin e një nënsistemi të shëndoshë nga një nënsistem tjetër i cili është duke dështuar.
- 29) **Mbrojtja dhe siguria** kërkojnë që sistemi të jetë në gjendje të dallojë shfrytëzuesit e tij. Shumica e sistemeve operative mirëmbajnë një listë të emrave të shfrytëzuesve dhe identifikuesin e shfrytëzuesit (user ID). Këto ID numerike janë unike, një për secilin shfrytëzues. Kur shfrytëzuesi logohet në sistem, faza e autentikimit përcakton ID-në e duhur të shfrytëzuesit për shfrytëzuesin. Ajo ID e shfrytëzuesit lidhet me të gjitha proceset dhe thredat e shfrytëzuesit. Kur një ID duhet të jetë e lexueshme nga një shfrytëzues, ID përkthehet në emër të shfrytëzuesit përmes listës së emrave të shfrytëzuesve.
- 30) **Virtualizimi** është një teknologji që mundëson të fshehim (abstrahojmë) harduerin e një kompjuteri (CPU-në, memorien, disqet, kartelat e rrjetës dhe pjesë tjera) në disa ambiente të ndryshme të ekzekutimit, duke krijuar në atë mënyrë iluzionin që secili ambient punon në kompjuterin e vet.

Ligj. 2

- 31) Një sistem operativ ofron ambientin në të cilin ekzekutohen programet. Ofron disa shërbime për programet dhe shfrytëzuesit e tyre. Shërbimet që ofrohen dallojnë nga njëri sistem operativ në një tjetër, por mund të identifikojmë disa grupe të shërbimeve.
- 32) Një pjesë e shërbimeve të sistemit operativ ofron funksione që ndihmojnë shfrytëzuesin.
- Interfejsi i shfrytëzuesit** - Pothuajse të gjitha sistemet operative kanë një interfejs të shfrytëzuesit (UI). Mund të jetë Command-Line (CLI), Graphics User Interface (GUI), touch-screen, Batch
 - Ekzekutimi i programit** - Sistemi duhet patjetër të jetë në gjendje të vendosë një program në DRAM, të ekzekutojë atë program, përfundojë ekzekutimin, qoftë në mënyrë të rregullt ose në mënyrë jo të rregullt (duke treguar gabimin)
 - Operacionet e H/D** - Një program që është duke u ekzekutuar mund të ketë nevojë për H/D, që mund të përfshijë një fajll ose pajisje H/D
 - Puna me sistemin e fajllave** - Sistemi i fajllave është i një rëndësie të veçantë. Programet kanë nevojë të lexojnë dhe shkruajnë fajlla, direktorime, krijojnë dhe fshijnë ato, kërkojnë ato, listojnë informatat e fajllit, menaxhimi i të drejtave.
 - Komunikime** - Proceset mund të shkëmbejnë informacione, në të njëjtin kompjuter ose në mes të kompjuterëve në një rrjetë. Komunikimi mund të bëhet duke përdorë një

memorie të përbashkët ose përmes bartjes së mesazheve (paketat barten nga sistemi operativ)

- f. **Gjetja e gabimeve** - sistemi operativ duhet të jetë në dijeni vazhdimisht për mundësinë e gabimeve. Mund të paraqiten në CPU dhe në harduerin e memories, në pajisje H/D, në programin e shfrytëzuesit. Për secilin lloj të gabimit, sistemi operativ duhet të ndërmarrë veprimin e duhur për të siguruar përdorim të drejtë dhe konsistent të kompjuterit. Mundësitë për debugging mund të rrisin në mënyrë të madhe aftësitë e shfrytëzuesit dhe programuesit për të përdorë në mënyrë efikase sistemin
 - g. Një pjesë tjetër e funksioneve të sistemit operativ ekzistojnë për të siguruar punën efikase të vet sistemit operativ përmes përdorimit të përbashkët të resurseve: **Dhënia e resurseve** - Kur shumë shfrytëzues ose shumë punë, që janë duke u ekzekutuar në mënyrë konkurente, resurset duhet t'u jepen secilave prej tyre 4 Shumë resurse - ciklet e CPU-së, memoria DRAM, memoria e fajllave, pajisjet H/D.
 - h. **Logimi** - Për të mbajtur shënime se sa shumë mund të përdorin shfrytëzuesit dhe çfarë lloj të resurseve
 - i. **Mbrojtja dhe siguria** - Krijuesit e informatave të ruajtura në një kompjuter me shumë shfrytëzues ose në një sistem kompjuterik të lidhur në rrjetë mund të kenë nevojë për të kontrolluar përdorimin e atij informacioni, proceset konkurente nuk duhet të ndërhyjnë në njëri-tjetrin 4 Mbrojtja përfshinë sigurimin se të gjitha qasjet në resurset e sistemit janë të kontrolluara 4 Siguria e sistemit nga palët e jashtme kërkon autentikimin e shfrytëzuesit, vlenë edhe për pajisjet e jashtme H/D për tentimet për qasje jo të lejuara.
- 33) Ekzistojnë disa mënyra për shfrytëzuesit që të përdorin sistemin operativ. Janë tre qasje themelore. Njëra ofron një interfejs përmes komandave, ose interpretues të komandave, që lejon shfrytëzuesit që të fusin direkt komanda për t'u ekzekutuar nga sistemi operativ. Dy qasjet tjera ia mundësojnë shfrytëzuesit të përdorë sistemin operativ përmes interfejsit grafik të shfrytëzuesit ose GUI.
- 34) **Interpretuesi i komandave** - Shumica e sistemeve operative, përfshirë Linux, UNIX dhe Windows, interpretuesin e komandave e trajtojnë si një program të veçantë që ekzekutohet kur të fillojë një proces ose kur një shfrytëzues logohet për herë të parë në sistem (në sistemet interaktive). Në sistemet me shumë interpretues të komandave, interpretuesit njihen si shell. Funksioni kryesor i një interpretuesi të komandave është të lexojë dhe ekzekutojë komandën e radhës të dhënë nga shfrytëzuesi. Shumë nga komandat të dhëna në këtë nivel punojnë me fajlla: krijo (create), fshij (delete), listo (list), printo (print), kopjo (copy), ekzekuto (execute) e tjera.
- 35) **Thirrjet sistimore** ofrojnë një interfejs (mundësi) për përdorimin e shërbimeve që ofrohen nga një sistem operativ. Këto thirrje zakonisht janë në formë të funksioneve të shkruara në gjuhët C dhe C++, edhe pse disa punë të nivelit më të ulët (për shembull, punët ku hardueri duhet të qaset direkt) mund të jetë e nevojshme të shkruhen përmes instruksioneve në gjuhën assembler.
- 36) **API** - Zhvilluesit e aplikacioneve dizajnojnë programet sipas një interfejsi të programimit (API). API specifikon një bashkësi të funksioneve që janë në dispozicion për një programues të aplikacioneve, përfshirë parametrat që i barten secilit funksion dhe vlerat e kthimit që mund të presë programuesi. Tre prej API-ve më të zakonshëm që janë në dispozicion të programuesve të aplikacioneve janë Windows API për sistemet Windows, POSIX API për sistemet Posix (që përfshijnë pothuajse të gjitha versionet e UNIX, Linux dhe macOS) dhe Java API për programet

që ekzekutohen në makinën virtuale të Java. Një programues i qaset një API përmes një librerie me kod që ofrohet nga sistemi operativ. Funksionet brenda një API zakonisht i thërrasin thirrjet sistemo për programuesin e aplikacioneve. Për shembull, funksioni `CreateProcess ( )` në Windows (që përdoret për të krijuar një proces të ri) e thërret thirrjen sistemo `NTCreateProcess ( )` në kernelin e Windows-it.

37) **Rrethina në kohën e ekzekutimit** (run-time environment – RTE) – e tërë bashkësia e softuerëve që nevojitet për të ekzekutuar programet e shkruara në një gjuhë programuese, përfshirë kompajlerët ose interpretuesit si dhe softuerë tjerë, si libraritë dhe loaders. RTE ofron një system-call interface (interfejs për thirrjet sistemo) që shërben si lidhje për te thirrjet sistemo që ofrohen nga sistemi operativ.

38) **Thirrjet sistemo** mund të ndahen në gjashtë kategori kryesore: kontroll i procesit, menaxhim i fajllave, menaxhim i pajisjeve, mirëmbajtje e informacionit, komunikime dhe mbrojtje. Figura 2.8 përmbledhë llojet e thirrjeve sistemo që mund të ofrohen nga një sistem operativ.

- a. Kontroll procesi
 - i. krijo procesin, përfundo procesin
 - ii. vendose në memorie, ekzekuto
 - iii. lexo atributet e procesit, vendos atributet e procesit
 - iv. ngjarje e pritjes, ngjarje e sinjalit
 - v. dhënia dhe lirim i memories
- b. Menaxhimi i fajllave
 - i. krijo fajllin, fshij fajllin
 - ii. hape, mbylle
 - iii. lexo, shkruaj, rivendose
 - iv. lexo atributet e fajllit, vendos atributet e fajllit
- c. Menaxhimi i pajisjeve
 - i. merr në shfrytëzim pajisjen, liro pajisjen
 - ii. lexo, shkruaj, rivendose
 - iii. lexo atributet e pajisjes, vendos atributet e pajisjes
 - iv. vendos ose largo pajisjet prej sistemit
- d. Mirëmbajtja e informacionit
 - i. lexo kohën ose datën, vendos kohën ose datën
 - ii. lexo të dhënat e sistemit, vendos të dhënat e sistemit
 - iii. lexo atributet e procesit, fajllit ose pajisjes
 - iv. vendos atributet e procesit, fajllit ose pajisjes
- e. Komunikimet
 - i. krijo, fshij lidhjen e komunikimit
 - ii. dërgo, prano mesazhet nëse përdoret modeli me bartje të mesazheve të emri i hostit ose emri i procesit
 - 1. nga klienti te serveri
 - iii. modeli me memorie shared krijo dhe merr qasjen në pjesët e memories
 - iv. bart informacionin e statusit
 - v. shto ose largo pajisjet në distancë
- f. Mbrojtja
 - i. kontrolli i qasjes në proces

- ii. lexo të drejtat mbi fajllin
- iii. vendos të drejtat mbi fa-jllin

39) Përmbajtja e memories së programit shkruhet në një fajll të veçantë në disk dhe mund të kontrollohet nga një debugger – një program i sistemit i dizajnuar për të ndihmuar programuesin në gjetjen dhe përmirësimin e gabimeve logjike ose bugs – për të përcaktuar rrënjët e problemit. Në rrethana të rregullta ose jo të rregullta, sistemi operativ duhet patjetër të bartë kontrollin të interpretuesit i komandave thirrës.

40) **Komunikimi mes proceseve:** Janë dy mënyra të zakonshme të komunikimit në mes të proceseve: modeli me shkëmbim të mesazheve dhe modeli me memorie të përbashkët. Shkëmbimi i mesazheve është i dobishëm për shkëmbimin e sasive të vogla të të dhënave sepse nuk është e nevojshme të shmanget ndonjë konflikt. Ky model implementohet më lehtë se modeli me memorie të përbashkët për komunikime në mes të kompjuterëve. Modeli me memorie të përbashkët mundëson shpejtësi maksimale të bartjes dhe lehtësi komunikimi, meqë mund të bëhet në nivelin e shpejtësisë së bartjes të memories kur ndodhë në një kompjuter. Megjithatë, ekzistojnë disa probleme sa i përket mbrojtjes dhe sinkronizimit në mes të proceseve që përdorin memorien e njëjtë.

41) **Programet e sistemit** ofrojnë një ambient të përshtatshëm për zhvillimin dhe ekzekutimin e programit. Mund të ndahen në:

- a. Menaxhimi i fajllave - Krijë, fshij, kopjo, riemërto, printo, dump (programi zhvendoset prej DRAM në ssd ose hdd), listo dhe në përgjithësi puna me fajlla dhe direktorime
- b. Informacioni i statusit
 - i. Disa kërkojnë nga sistemi informata si - data, koha, madhësia e memories, hapësira e diskut, numri i shfrytëzuesve
 - ii. Tjerët ofrojnë informata të detajuara të performancës, logimit dhe për debugging
 - iii. Zakonisht, këto programe formatojnë dhe printojnë daljen në ekran ose në ndonjë pajisje tjetër dalëse
 - iv. Disa sisteme implementojnë një registry - që përdoret për të ruajtur dhe lexuar informata të konfigurimit
- c. Ndryshimi i fajllave
 - i. Editorë të tekstit për krijim dhe ndryshim të fajllit
 - ii. Komanda speciale për kërkim në përmbajtje të fajllave ose për të realizuar ndryshime në tekst
- d. Përkrahja për gjuhë programuese - ofrohen kompajlerë, assemblerë, debuggers dhe interpretues
- e. Vendosja e programit në memorie dhe ekzekutimi – loaders absolut, loaders të rinvendosshëm, linkage editors dhe overlayloaders, debugging i sistemeve për një nivel më lart dhe gjuha e makinës
- f. Komunikimet - Ofrojnë mekanizma për krijim të lidhjeve virtuale në mes proceseve, shfrytëzuesve dhe sistemeve kompjuterike
 - i. Lejojnë shfrytëzuesit të shkëmbejnë mesazhe, shfletojnë ueb faqe, dergojnë e-mail, logim në distancë, dërgim të fajllave nga një kompjuter në një tjetër
- g. Shërbimet në background
 - i. Fillojnë gjatë startimit të sistemit

- ii. Disa vetëm për të ndihmuar në startimin e sistemit, pastaj përfundojnë 4 Disa që nga startimi i sistemit deri në ndaljen e tij
 - iii. Ofrojnë mundësi si kontrollimi i diskut, planifikimi i proceseve, mbajtja e shënimeve për gabimet, printimi
 - iv. Ekzekutohen në mënyrën e shfrytëzuesit jo në atë të kernelit
 - v. Të njohura si shërbime, nësisteme, daemons
 - h. Programet aplikative
 - i. Nuk janë pjesë përbërëse e sistemit
 - ii. Ekzekutohen me kërkesë të shfrytëzuesit
 - iii. Zakonisht nuk konsiderohen pjesë e sistemit operativ
 - iv. Thirren përmes linjës së komandave, klikimeve me maus, prekjeve me gisht
- 42) **Linker & Loader** - Kodi burimor i kompajluar në fajll binar që mund të vendoset në memorien DRAM – relocatable object file. Linker i kombinon këta fajlla në një fajll të vetëm executable - përdorë edhe libraritë. Programi është në një memorie sekondare si binary executable, duhet patjetër të vendoset në DRAM nga loader për t'u ekzekutuar. Relocation ua vendosë adresat përfundimtare pjesëve të programit dhe rregullon kodin e të dhënat në program që të përputhen me ato adresa. Sistemet moderne me qëllim të përgjithshëm nuk i lidhin libraritë në fajlla të ekzekutueshëm. Për dallim, dynamically linked libraries (në Windows, DLL) vendosen në memorie sipas nevojës, përdoret nga të gjitha programet që përdorin të njëjtin version të librarisë (që vendoset vetëm njëherë në memorie). Fajllat e ekzekutueshëm kanë një format standard, ashtu që sistemi operativ e di si t'i vendosë ata në memorie dhe ekzekutojë.
- 43) Programet që kompajlohen për një sistem operativ zakonisht nuk mund të ekzekutohen për një sistem operativ tjetër. Secili sistem operativ ofron thirrjet e veta sistimore - Formatin e fajllit, etj. Programet mund të shkruhen për shumë sisteme operative
- a. Shkruar me një gjuhë të interpretuar si Python, Ruby dhe ekziston nga një interpretues për shumë sisteme operative
 - b. Programi i shkruar në një gjuhë që përfshinë një VM që përmban programin që është duke u ekzekutuar (si për shembull Java)
 - c. Përdorimi i një gjuhë standarde (si për shembull C), kompajlim i programit për secilin sistem operativ në mënyrë që programi të mund të ekzekutohet
- 44) Ekzistojnë edhe sfida tjera në nivelet më të ulëta në sistem, duke përfshirë këto:
- a. Çdo sistem operativ e ka një format binar për aplikacionet që përcakton shtrirjen e pjesës header, instruksioneve dhe variablave.
 - b. CPU-të kanë bashkësi të ndryshme të instruksioneve dhe vetëm ato aplikacione që kanë instruksionet e duhura (të lejuara) mund të ekzekutohen.
 - c. Sistemet operative ofrojnë thirrje sistimore që dallojnë shumë në shumë aspekte në sisteme operative të ndryshme, përfshirë operandët e caktuar dhe mënyrën e radhitjes së operandëve që përdoren, si një aplikacion i thërret thirrjet sistimore, numërimi i tyre dhe numri, kuptimet e tyre dhe kthimi i rezultateve prej tyre.
- 45) **Application Binary Interface (ABI)** është një arkitekturë e ngjashme me API, përcakton se si pjesë të ndryshme të kodit mund të komunikojnë me sistemin operativ në një arkitekturë të caktuar, CPU, etj.
- 46) **Dizajni dhe implementimi** i sistemit operativ nuk është “ i zgjidhshëm”, por disa qasje janë vërtetuar si të sukseshme. Struktura e brendshme e shumë sistemeve operative dallon. Fillimi i

dizajnimit duke përcaktuar qëllimet dhe specifikat, dhe ndikohet nga përzgjedhja e harduerit, lloji i sistemit. Qëllimet e shfrytëzuesit dhe qëllimet e sistemit

- a. Qëllimet e shfrytëzuesit – sistemi operativ duhet të jetë i përshtatshëm për përdorim, i lehtë për t'u mësuar, i besueshëm, i sigurtë dhe i shpejtë
- b. Qëllimet e sistemit – sistemi operativ duhet të jetë i lehtë për t'u dizajnuar, implementuar, mirëmbajtur, si dhe fleksibil, i besueshëm, pa gabime dhe efikas

47) Specifikimi dhe dizajni i një sistemi operativ është një punë mjaft kreative e inxhinierisë softuerike.

48) **Teknika – Mekanizmi:** Një parim me rëndësi është ndarja e teknikës prej mekanizmit.

Mekanizmat përcaktojnë si të bëhet ndonjë punë; teknikat (politikat) përcaktojnë çka duhet bërë. Për shembull, konstrukti i tajmerit (shikoni në Figura 1.4.3) është një mekanizëm për të siguruar mbrojtjen e CPU-së, por vendimi se për sa gjatë duhet të vendoset vlera e tajmerit është një vendim i teknikës. Ndarja e teknikës prej mekanizmit është një parim shumë i rëndësishëm, lejon fleksibilitet maksimal nëse ndryshohen vendimet e teknikës.

49) Sistemet operative të hershme ishin shkruar në gjuhën assembler. Sot, shumica shkruhen në një gjuhë të lartë programimi si C ose C++, me pjesë të vogla të sistemit të shkruara në gjuhën assembler. Përparësitë e përdorimit të një gjuhe programuese të lartë, ose së paku një gjuhe për implementim të sistemeve, për implementim të sistemeve janë të njëjta me ato që fitohen kur gjuha përdoret për zhvillimin e programeve aplikative: kodi mund të shkruhet më shpejtë, është më kompakt dhe është më i lehtë për t'u kuptuar dhe për debugging. Mangësia e vetme e mundshme e implementimit të një sistemi operativ në një gjuhë programuese të lartë janë shpejtësia e zvogëluar dhe rritja e kërkesave për hapësirë të ruajtjes.

50) **Struktura monolitike** - UNIX – i kufizuar nga mundësitë e harduerit, sistemi operativ UNIX kishte një strukturim të kufizuar, ashtu që i tërë funksionaliteti i kernelit do të vendosej në një fajll binar që ekzekutohet në një hapësirë të vetme të adresave. Sistemi operativ UNIX përbëhet nga dy pjesë të ndara

- a. Programet e sistemit
 - b. Kerneli përbëhet nga çdo gjë që ndodhet përfundimisht në interfejsin e thirrjes së sistemit dhe harduerit. Ofron sistemin e fajllave, planifikimin e CPU-së, menaxhimin e memories dhe funksione tjera të sistemit operativ; një numër shumë i madh i funksioneve në një nivel
- Kernelët monolitik kanë një përparësi dalluese të performancës: ka pak punë që duhen kryer në interfejsin e thirrjeve të sistemit dhe komunikimi brenda në kernel është i shpejtë.

51) **Qasja Shtresore** - Sistemi operativ ndahet në disa shtresa (nivele), secila e ndërtuar mbi nivelin poshtë saj. Shtresa e poshtme (shtresa 0), është hardueri; shtresa më e lartë (shtresa N) është interfejsi i shfrytëzuesit. Përmes modularitetit, shtresat përzgjedhen ashtu që secila përdorë funksionet (operacionet) dhe shërbimet e vetëm shtresës që ndodhet menjëherë poshtë saj. Përparësia kryesore e qasjes me shtresa është thjeshtësia në ndërtim dhe debugging, ashtu që ndryshimet në një pjesë kanë ndikim vetëm në atë pjesë dhe jo në pjesët tjera, duke u lejuar implementuesve të sistemit më shumë liri në krijimin dhe ndryshimin e mënyrës se si sistemi funksionon nga brenda.

52) **Mikrokernel** - Kernelin të ndarë në module përmes qasjes me mikrokernel - Kjo metodë e strukturimit të sistemit operativ duke larguar të gjitha pjesët jo thelbësore prej kernelit në hapësirë të ndryshme të adresave. Funksioni kryesor i mikrokernelit është të ofrojë komunikim në mes të

programit të klientit dhe disa shërbimeve të cilat poashtu janë duke u ekzekutuar në hapësirën e shfrytëzuesit. Komunikimi bëhet përmes bartjes së mesazheve. Perparesite:

- a. E lehtë për të zgjeruar një mikrokernel
- b. E lehtë për të bartur sistemin operativ në arkitektura të reja
- c. Më i besueshëm (më pak kod ekzekutohet në mënyrën e kernelit)
- d. Më i sigurtë

Mangësitë: Duhet kryer disa punë shtesë për komunikimin në mes të hapësirës së shfrytëzuesit dhe hapësirës së kernelit.

- 53) Shumë sisteme operative implementojnë moduleve të kernelit të vendosshme në memorie - loadable kernel modules (LKMs). Te ky dizajn, kerneli ka një bashkësi të pjesëve themelore dhe mund të lidhë (përdorë) shërbime shtesë përmes moduleve, gjatë kohës së startimit të sistemit ose gjatë kohës së ekzekutimit. Përdorë qasjen e orientuar në objekte, çdo pjesë esenciale është e ndarë. Secila komunikon me tjerat përmes interfejsëve të përcaktuar mire, dhe secila mund të vendoset në memorie sipas nevojës nga kernel. Rezultati përfundimtar do të ishte një sistem me shtresa në të cilin secila pjesë e kernelit ka interfejsë të përcaktuara dhe të mbrojtura mirë; por është më fleksibil se një sistem me shtresa, sepse secili modul mund të thërrasë cilindo modul tjetër.
- 54) Shumica e sistemeve operative nuk janë një sistem i pastër. Hibridi kombinon shumë qasje për të adresuar performacën, sigurinë dhe nevojat për përdorim. Kernelet e Linux dhe Solaris në hapësirën e adresave të kernelit, monolitike, plus modulare për vendosje dinamike të funksionalitetit. Në memorie Windows-i është kryesisht monolitik, plus mikrokernel për personalitetet e shumë nënsistemeve.
- 55) Sistemet operative dizajnohen që të ekzekutohen në një klasë të kompjuterëve që kanë konfigurime të ndryshme të pajisjeve periferike. Për të ndërtuar një sistem operativ nga fillimi, duhet patjetër të ndjekën këta hapa:
- a. Shkrimi i kodit të sistemit operativ (ose marrja e një kodi që kanë shkruar tjerët).
 - b. Konfigurimi i sistemit operativ për sistemin në të cilin do të ekzekutohet.
 - c. Kompajlimi i sistemit operativ (system build).
 - d. Instalimi i sistemit operativ.
 - e. Startimi i kompjuterit dhe sistemit operativ të ri.
- 56) Procesi i startimit të një kompjuteri duke vendosur kernelin në memorie njihet si startimi (booting) i sistemit. Në shumicën e sistemeve, startimi bëhet kështu:
- a. Një program i vogël i njohur si bootstrap ose boot loader e gjenë kernelin.
 - b. Kerneli vendoset në DRAM dhe startohet.
 - c. Kerneli e starton harduerin.
 - d. Sistemi i fajllave bëhet pjesë e sistemit.
- 57) Një program i vogël – bootstrap loader, BIOS, që ruhet në ROM ose EEPROM e gjen kernelin, e vendosë atë në memorie dhe e starton atë. Ka raste kur kjo bëhet me dy hapa ku boot block në një vend fiks vendoset në memorie nga një program në ROM, i cili e vendosë në DRAM bootstrap loader nga disku. Sistemet moderne e zëvendësojnë BIOS-in me Unified Extensible Firmware Interface (UEFI). Një bootstrap loader i zakonshëm, GRUB, lejon përzgjedhjen e kernelit prej shumë disqeve, versioneve, opcioneve të kernelit. Kernel vendoset në DRAM dhe

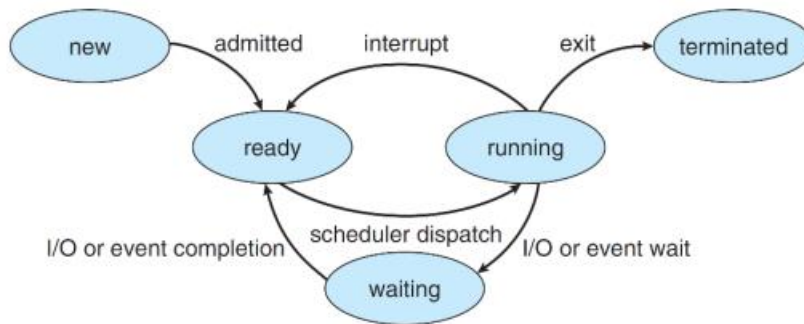
sistemi ekzekutohet. Boot loaders zakonisht lejojnë shumë gjendje të startimit, si për shembull modi me vetëm një shfrytëzues

- 58) GRUB është një program bootstrap me kod të hapur për sistemet Linux dhe UNIX. Vlerat e parametrave të startimit për sistemin vendosen në një fajll konfigurimi të GRUB, i cili vendoset në memorie gjatë startimit. GRUB është fleksibil dhe lejon të bëhen ndryshime gjatë kohës së startimit, përfshirë ndryshimin e parametrave të kernelit dhe madje edhe zgjedhjen në mes të kernelëve të ndryshëm që mund të startohen
- 59) Në fund, boot loaders për shumicën e sistemeve operative - përfshirë Windows, Linux dhe macOS, si dhe iOS dhe Android - ofrojnë mundësinë që të startohet në recovery mode ose single-user mode për të testuar në lidhje me probleme të harduerit, përmirësimi i gabimeve në sisteme të fajllave e madje edhe riinstalimin e sistemit operativ. Përveç dështimeve të harduerit, sistemet kompjuterike mund të përballen edhe me gabime në softuer dhe performancë të dobët të sistemit operativ.
- 60) Debugging është gjetja dhe përmirësimi i gabimeve logjike ose bugs. Sistemet operative krijojnë log files që përmbajnë informata për gabime. Dështimi i një aplikacioni mund të krijojë një fajll core dump në të cilin ruhet përmbatja e memories së procesit. Dështimi i sistemit operativ mund të krijojë një fajll crash dump që përmban përmbajtjen e memories së kernelit. Përtej dështimeve, performance tuning mund të optimizojë performancën e sistemit. Ndonjëherë trace listings të aktiviteteve, ruajtje e analizave. Profiling është përdorim i regjistrimit instruction pointer për të vërejtur trende statistikore

Ligj. 3

- 61) **Procesi** është një program në ekzekutim. Një proces paraqet një njësi të punës brenda një sistemi kompjuterik modern.
- 62) Shtrirja e memories të një procesi zakonisht ndahet në shumë pjesë
- Aktiviteti aktual përfshirë program counter, regjistrat e procesorit
 - Pjesa e tekstit - kodi i ekzekutueshëm
 - Pjesa e të dhënave - variablat globale
 - Pjesa e heap - memoria që jepet në mënyrë dinamike për shfrytëzim gjatë ekzekutimit të programit
 - Pjesa e stekut - hapësirë për ruajtje të përkohshme gjatë thirrjes së funksioneve (si parametrat e funksionit, adresat e kthimit dhe variablat lokale)
- 63) Një program bëhet proces kur një program i ekzekutueshëm të vendoset në DRAM. Dy mënyrat e zakonshme për vendosjen në DRAM të fajllave të ekzekutueshëm janë klikimi i dyfishtë në ikonën që paraqet fajllin e ekzekutueshëm dhe futja e emrit të fajllit të ekzekutueshëm përmes linjës me komanda (si prog.exe ose a.out).
- 64) **Gjendja e procesit** - Gjatë ekzekutimit, procesi ndërron gjendjen. Gjendja e procesit përcaktohet pjesërisht nga aktiviteti aktual i procesit. Një proces mund të jetë në njëren prej këtyre gjendjeve:
- New (i sapo krijuar). Procesi u krijua.
 - Running (duke u ekzekutuar). Instruksionet janë duke u ekzekutuar.
 - Waiting (në pritje). Procesi është duke pritur për paraqitjen e ndonjë ngjarjeje (si përfundimi i ndonjë operacioni të H/D ose pranimin e një sinjali).
 - Ready (gati). Procesi është duke pritur që të marrë CPU-në në shfrytëzim.

e. Terminated (i përfunduar). Procesi ka mbaruar ekzekutimin.



65) **PCB** - Çdo proces në sistem operativ paraqitet me një bllok të kontrollit të procesit (PCB) – i quajtur edhe si blloku për kontroll të punës. PCB përmban shumë pjesë të informacionit që lidhen me një proces, duke përfshirë këto:

- Gjendja e procesit – running, waiting, etj.
- Program counter – vendi i instruksionit që duhet të ekzekutohet me radhë
- Regjistrat e CPU-së – përmbajtjet e të gjithë regjistrave që lidhen me procesin
- Informacioni i planifikimit të CPU-së - prioritetet, planifikimi i pointerëve (adresave) për queue
- Informacioni për menaxhim të memories – memoria që i është dhënë procesit, vlerat e regjistrave bazë dhe të kufirit dhe tabelat e faqes, ose tabelat e segmenteve
- Informacioni i përdorimit – përdorimi i CPU-së, koha e kaluar që nga startimi, kufizime kohore, numrat e kontove (account), numrat e punëve ose proceseve e të tjera
- Informacioni i gjendjes për H/D – pajisjet H/D që i janë dhënë procesit, lista e fajllave të hapur etj.

PCB thjeshtë shërben si një vend për ruajtjen e të dhënave që nevojiten për të startuar, ristartuar një proces, bashkë me informacionin për ekzekutimin.

66) **Planifikuesi i procesit** zgjedhë në mes të proceseve për ekzekutimin e radhës në bërthamën e CPU-së. Numri i proceseve që janë në memorie njihet si **shkalla e multiprogramimit**.

67) Kur proceset të krijohen, ato vendosen në një listë të proceseve që janë gati, ku ato janë gati dhe duke pritur ekzekutimin në një bërthamë të CPU-së. Proceset që janë duke pritur që të paraqitet ndonjë ngjarje e caktuar – si përfundimi i punës në H/D – vendosen në një listë të pritjes.

68) Një proces i ri fillimisht vendoset në listën gati. Ai pret aty derisa të përzgjedhet (dispatched) për ekzekutim. Kur një procesi t'i jetë dhënë një bërthamë e CPU-së dhe është duke u ekzekutuar, një prej disa ngjarjeve do të mund të paraqitet:

- Procesi do të mund të paraqiste një kërkesë për H/D dhe pastaj të vendosej në një listë të pritjes.
- Procesi do të mund të krijonte një proces të ri fëmijë dhe të vendosej në një listë të pritjes përderisa pret përfundimin e procesit fëmijë.
- Procesi do të mund të detyrohej të largohej prej bërthamës, prej ndonjë ndërprerjeje ose pasi t'i ketë kalur koha e tij dhe të vendosej në listën gati

69) **Swapping** - Disa sisteme operative kanë një formë të ndërmjetme të planifikimit, e njohur si swapping (ndërrimi i vendit), ideja bazë e të cilës është se ndonjëherë mund të ketë përparësi

largimi i një procesi prej memories dhe ashtu të zvogëlohet shkalla e multiprogramimit. Më vonë, procesi mund të rivendoset në memorie dhe ekzekutimi i tij mund të vazhdojë aty ku ishte ndërprerë. Ndodh kur tejmbushet DRAM.

- 70) **Context Switch** - Kur CPU kalon në një process tjetër, sistemi duhet patjetër të ruaj gjendjen e procesit të vjetër dhe të vendosë në regjistra gjendjen e ruajtur për procesin e ri përmes ndërrimit të kontekstit. Konteksti paraqitet në PCB-në e procesit. Konteksti përshinë vlerat e regjistrave të CPU-së, gjendjen e procesit dhe informacionin për menaxhimin e memories.
- 71) Një ndërrim konteksti kërkon thjeshtë ndërrimin e pointerit për në bashkësinë aktuale të regjistrave. Natyrisht, nëse ka më shumë procese aktive se sa bashkësi të regjistrave, sistemi kopjon të dhënat e regjistrave prej dhe në memorie, sikur më pare.
- 72) Procesi krijues quhet procesi prind dhe proceset e reja quhen fëmijët e atij procesi. Secili prej këtyre proceseve të reja mund të krijojë procese tjera, duke formuar një pemë të proceseve. Shumica e sistemeve operative (përfshirë UNIX, Linux dhe Windows) identifikojnë proceset përmes një identifikuesi të procesit (ose pid) unik, që zakonisht është një numër i plotë. Pid ofron një vlerë unike për çdo proces në sistem dhe mund të përdoret si indeks për t'u qasur attributeve të ndryshme të procesit në kernel.
- 73) Kur një proces të krijojë një proces, ekzistojnë dy mundësi për ekzekutim:
- Prindi vazhdon të ekzekutohet në mënyrë konkurente me fëmijët e tij.
 - Prindi pret derisa disa ose të gjithë fëmijët të kenë përfunduar.
- Janë dy mundësi të hapësirës së adresave për procesin e ri:
- Procesi fëmijë është një kopje e procesit prind (ai ka të njëjtin program dhe të dhënat sikur prindi).
 - Procesi ka një program të ri të vendosur brenda tij.
- 74) Mundësitë për shfrytëzim të përbashkët:
- Prindi dhe fëmijët përdorin bashkë të gjitha resurset
 - Fëmijët përdorin një pjesë të resurseve të prindit
 - Prindi dhe fëmijët nuk përdorin bashkë resurset
- 75) Hapësira e adresave:
- Fëmijët kanë hapësirën e njëjtë të prindit
 - Fëmija ka një program brenda tij
- 76) Një proces përfundon kur të kryejë ekzekutimin e shprehjes së fundit të tij dhe kërkon nga sistemi operativ që të fshijë atë përmes thirrjes sistimore exit (). Në atë moment, procesi mund të kthejë një vlerë të gjendjes (zakonisht një numër i plotë) të procesit prind që e pret atë (përmes thirrjes sistimore wait ()). Të gjitha resurset e procesit – përfshirë memorien fizike dhe memorien virtuale, fajllat e hapur dhe baferët e H/D – merren dhe kthehen nga sistemi operativ.
- 77) Një prind mund të përfundojë ekzekutimin e njërit prej fëmijëve të tij për disa arsye, si këto:
- Fëmija ka tejkaluar shfrytëzimin e disa prej resurseve që i janë dhënë. (Për të përcaktuar nëse kjo ka ndodhur, prindi duhet patjetër të ketë një mekanizëm për të kontrolluar gjendjen e fëmijëve të tij).
 - Detyra që i ishte dhënë fëmijës nuk nevojitet më.
 - Prindi është duke përfunduar dhe sistemi operativ nuk lejon që një fëmijë të vazhdojë nëse prindi i tij përfundon.

- 78) Një proces që ka përfunduar, por prindi i të cilit ende nuk ka thirrur wait (), njihet si proces zombie. Një process prind nese ka përfunduar, por nuk ka thirrur wait (), atehere ai ka lene proceset femije jetim (orphan). Per kete, ne UNIX perdoret init, Linux – systemd.
- 79) Hierarkia e proceseve ne Android - Nga më i rëndësishmi te më pak i rëndësishmi, hierarkia e klasifikimeve të proceseve është si vijon:
- Procesi foreground - Procesi aktual që shihet në ekran, që përfaqëson aplikacionin të cilin është duke përdorë shfrytëzuesi.
 - Procesi i dukshëm - Një proces që nuk shihet direkt në foreground (paravijë), por që është duke kryer një punë të cilës i referohet procesi në foreground (pra, një proces që kryen një punë gjendja e të cilës paraqitet në procesin foreground).
 - Procesi i shërbimit - Një proces që është i ngjashëm me një proces në background (prapavijë), por që është duke kryer një punë që shihet nga shfrytëzuesi (si dëgjimi i muzikës).
 - Procesi background - Një proces që mund të kryejë ndonjë punë, por që nuk shihet nga shfrytëzuesi.
 - Procesi i zbrazët – Një proces që nuk ka ndonjë pjesë aktive të lidhur me ndonjë aplikacion
- Android do të fillojë përfundimin e atyre proceseve që janë më pak të rëndësishëm.
- 80) Proceset që ekzekutohen në mënyrë konkurente në sistem operativ mund të jenë të pavarura ose bashkëpunuese. Një proces është i pavarur nëse ai nuk i ndanë të dhënat me ndonjë proces tjetër që është duke u ekzekutuar në sistem. Një proces është bashkëpunues nëse ai ndikon ose mund të ndikohet nga proceset tjera që janë duke u ekzekutuar në sistem. Thjeshtë, çdo proces që ndanë të dhënat me proceset tjera është një proces bashkëpunues.
- 81) Arsyet e proceseve bashkëpunuese:
- Përdorimi i përbashkët i informacionit
 - Shpejtimi i llogaritjes
 - Modulariteti
 - Lehtësia
- 82) **IPC** - Proceset bashkëpunuese kërkojnë një mekanizëm për komunikim në mes proceseve (interprocess communication – IPC) që do t'u mundësojë atyre të shkëmbejnë të dhëna – pra, të dërgojnë dhe pranojnë të dhëna njëri-tjetrit. Janë dy modele themelore të komunikimit në mes proceseve: memorie e përbashkët (me e shpejte) dhe bartje e mesazhit (per shkembime te te dhenave te vogla).
- 83) Komunikimi në mes proceseve duke përdorur memorien e përbashkët kërkon që proceset komunikuese të krijojnë një regjion të përbashkët në memorie.
- 84) Një prodhues prodhon informacion që konsumohet nga një proces konsumues. Për shembull, një kompajler mund të prodhojë kod në assembler që konsumohet nga një assembler. Një zgjidhje e problemit të prodhuesit-konsumuesit përdorë memorien e përbashkët. Për të mundësuar që proceset e prodhuesit dhe konsumuesit të ekzekutohen në mënyrë konkurente, duhet patjetër të jetë në dispozicion një bafer që mund të mbushet nga prodhuesi e të zbrazet nga konsumuesi. Ky bafer do të jetë në një regjion të memories së përbashkët që përdoret nga proceset e prodhuesit dhe konsumuesit. Dy mundësi:
- unbounded-buffer nuk e kufizon madhësinë e buffer-it:
 - Prodhuesi nuk pret asnjëherë

- ii. Konsumuesi pret nëse nuk ka ndonjë buffer për të konsumuar
- b. bounded-buffer pret që madhësia e buffer-it të jetë e kufizuar
 - i. Prodhuksi duhet patjetër të presë nëse të gjithë bufferët janë plotësisht të mbushur
 - ii. Konsumuesi pret nëse nuk ka ndonjë buffer për të konsumuar

```
#define BUFFER_SIZE 10
```

```
typedef struct {
    . . .
} item;
```

```
item buffer [BUFFER_SIZE];
int in = 0;
int out = 0;
```

85)

```
item next_produced;
```

```
while (true) {
    /* produce an item in next_produced */

    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */

    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
```

86)

```
}
```

Kodi I prodhuesit

```
item next_consumed;
```

```
while (true) {
    while (in == out)
        ; /* do nothing */

    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next_consumed */
```

87)

```
}
```

Kodi I konsumuesit

88) Proceset komunikojnë me njëri-tjetrin pa përdorë variablat e përbashkëta. Mundësia IPC ofron dy operacione: send (mesazhi) dhe receive (mesazhi). Madhësia e mesazhit është fikse ose e ndryshueshme. Nëse procest P dhe Q duan të komunikojnë, ato duhet patjetër të dërgojnë mesazhet te dhe pranojnë mesazhet nga njëri-tjetri: një lidhje komunikimi duhet patjetër të ekzistojë në mes tyre. Janë disa metoda për implementimin logjik dhe fizik të lidhjes dhe operacionet send () / receive ():

Fizike:

- a. Memorie e përbashkët
- b. Bus harduerik
- c. Rrjetë

Logjike:

- d. Komunikim direkt ose indirekt
- e. Komunikim sinkron ose asinkron
- f. Baferim automatik ose eksplisit

89) **Emertimi** - Duhet të ekzistojë një mënyrë se si proceset komunikuese t'i referohen njëra-tjetrës. Ato mund të përdorin komunikimin direkt ose indirekt.

90) **Komunikimi direkt**, secili proces që dëshiron të komunikojë duhet që në mënyrë eksplcite të emërtojë pranuesin ose dërguesin e komunikimit. Në këtë skemë, operacionet send () dhe receive () definohen si:

- a. send (P, mesazhi) - dërgo një mesazh procesit P.
- b. receive (Q, mesazhi) – prano një mesazh nga procesi Q.

Veçoritë e lidhjes së komunikimit

- c. Lidhjet realizohen në mënyrë automatike
- d. Një lidhje i shoqërohet dy proceseve komunikuese
- e. Në mes dy çifteve ekziston saktësisht një lidhje
- f. Lidhja mund të jetë një drejtimështe, por zakonisht është dy drejtimështe

91) Kjo skemë përdorë simetrinë në adresim; procesi dërgues dhe pranues duhet patjetër të përdorin emrin e tjetrit për komunikim. Një variant i kësaj skeme përdorë asimetri në adresim. Te ky variant, vetëm dërguesi përdorë emrin e pranuesit; nuk kërkohet që pranuesi të përdorë emrin e dërguesit. Në këtë skemë, operacionet send () dhe receive () definohen si në vijim: send (P, mesazhi) - dërgo një mesazh procesit P. receive (id, mesazhi) – prano një mesazh nga ndonjë proces. Variabla id e merrë vlerën e emrit të procesit me të cilin bëhet komunikimi. Mangësia e të dy këtyre skemave (simetrike dhe asimetrike) është modulariteti i kufizuar i definimeve të proceseve. Ndërrimi i identifikuesit të një procesi mund të paraqesë nevojën e shqyrtimit të definimeve të të gjithë proceseve.

92) **Komunikimi Indirekt** - Mesazhet dërgohen dhe pranohen përmes kutive postare (të njohura edhe si porte)

- a. Çdo kuti postare ka një id unike
- b. Proceset mund të komunikojnë vetëm nëse përdorin një kuti postare të përbashkët

93) Veçoritë e lidhjes së komunikimit:

- a. Lidhja realizohet vetëm nëse proceset përdorin kutinë e njëjtë postare
- b. Një lidhje mund t'u shoqërohet shumë proceseve • Çdo çift i proceseve mund të përdorë bashkë disa lidhje të komunikimit • Lidhja mund të jetë një drejtimështe ose dy drejtimështe

94) Operacionet e Sistemit Operativ:

- a. Krijimi i një kutie postare (porti)
- b. Dërgimi dhe pranimi i mesazheve përmes kutisë postare
- c. Fshirja e një kutie postare

95) Operacionet më elementare definohen si:

- a. send (A, mesazhi) – dërgimi i një mesazhi te kutia postare A
- b. receive (A, mesazhi) – pranimi i një mesazhi nga kutia postare A

96) **Sinkronizimi** - Bartja e mesazheve mund të jetë bllokuese ose jo bllokuese – e njohur edhe si sinkrone dhe asinkrone.

- a. Dërgim bllokues. Procesi dërgues bllokohet derisa mesazhi të pranohet nga pranuesi ose nga kutia postare.
 - b. Dërgim jo bllokues. Procesi dërgues dërgon mesazhin dhe vazhdon punën.
 - c. Pranim bllokues. Pranuesi bllokohet derisa mesazhi të jetë në dispozicion.
 - d. Pranim jo bllokues. Pranuesi pranon mesazhin ose vlerën null.
- 97) Kombinime të ndryshme të send () dhe receive () janë të mundshme. Kur send () dhe receive () janë bllokues, kemi një situatë rendezvous (takim me marrëveshje në të njëjtin vend dhe kohë) në mes dërguesit dhe pranuesit.
- 98) **Baferimi** - Kur komunikimi të jetë direkt ose indirekt, mesazhet e shkëmbyera nga proceset komunikuese qëndrojnë në një listë të përkohshme. Thjeshtë, këto lista mund të implementohen në tre mënyra:
- a. Kapaciteti zero (pa baferim) – asnjë mesazh nuk vendoset në queue të lidhjes. Dërguesi duhet patjetër të presë pranuesin (rendezvous)
 - b. Kapacitet i kufizuar (baferim automatik) – gjatësi fikse prej n mesazheve. Dërguesi duhet të presë nëse lidhja është mbushur plotësisht
 - c. Kapacitet i pa kufizuar (baferim automatik) – gjatësi e pafundme. Dërguesi asnjëherë nuk pret
- 99) Bartja e mesazheve përmes thirrjes të avancuar të procedurave lokale (LPC)
- a. Punon vetëm në mes proceseve në të njëjtin sistem
 - b. Përdorë portet (e lidhjes dhe komunikimit) (ngjashëm me kutitë postare) për të krijuar dhe mirëmbajtur kanalet e komunikimit

Komunikimi punon si në vazhdim:

- c. Klienti lidhet me objektin e portit të lidhjes
- d. Klienti dërgon një kërkesë për lidhje
- e. Serveri krijon dy porte komunikimi private dhe kthen lidhjen me njërin prej tyre te klienti
- f. Klienti dhe serveri përdorin lidhjen përkatëse me portin për të dërguar mesazhet ose për të pranuar përgjigjet

Ligj. 4

- 1) Thredi është një njësi themelore e përdorimit të CPU-së; thredi ka një ID të thredit, një regjistër PC, një bashkësi të regjistrave dhe një stek. Ai ndanë me thredat tjerë që i takojnë procesit të njëjtë pjesën e tij të kodit, pjesën e të dhënave dhe resurse tjera të sistemit operativ si fajllat e hapur dhe sinjalet.
- 2) Shumica e aplikacioneve moderne janë me shumë thread. Thredat ekzekutohen brenda aplikacionit, dhe shumë detyra në aplikacion mund të implementohen si threda të ndarë
 - a. Ndërrimi i pamjes në ekran
 - b. Leximi i të dhënave
 - c. Korrigjimi i gabimeve drejtshkrimore
 - d. Përgjigjja ndaj një kërkesë prej rrjetës
- 3) Krijimi i procesit është i rëndë përderisa krijimi i thredit është i lehtë. Mund të thjeshtojë kodin, rris efikasitetin. Kernelët janë në përgjithësi me shumë thread, si p.sh: thread per menaxhimin e pajisjeve, memories, dhe thread per nderprerjet etj.

- 4) Përfitimet nga programimi me shumë threda mund të ndahen në katër kategori kryesore:
 - a. Përgjigjja – mund të lejojë ekzekutim të vazhdueshëm nëse një pjesë e procesit bllokohet, e rëndësishme sidomos për interfejsin e shfrytëzuesit
 - b. Ndarja e resurseve – thredat përdorin në mënyrë të nënkuptuar resurset e procesit dhe memorien, më e lehtë se modeli me memorie të përbashkët ose me bartje të mesazheve
 - c. Ekonomia – më e lirë se krijimi i procesit, më pak punë dhe me pak kohe nevojitet për kalimin nga njëri thred në tjetrin se sa ndërrimi i kontekstit
 - d. Shkallëzueshmëria – përdorimi i shumë thredave në një arkitekturë me shumë procesorë, lejon thread-at të ekzekutohen në mënyrë paralele në bërthama të ndryshme
- 5) **Sistemet Multicore** - programimi me shumë threda ofron një mekanizëm për përdorim më eficient të këtyre bërthamave llogaritëse dhe konkurrencë të përmirësuar.
- 6) Një sistem konkurent përkrahë më shumë se një detyrë duke lejuar që të gjitha detyrat të bëjnë progres (përparim). Për dallim, një sistem paralel mund të kryejë më shumë se një detyrë në të njëjtën kohë. Kështu, është e mundur të kemi konkurrencë pa paralelizëm.
- 7) Në përgjithësi, pesë aspekte paraqesin sfida në programimin për sisteme me shumë bërthama:
 - a. Identifikimi i detyrave - kontrollimin e aplikacioneve për të gjetur pjesë që mund të ndahen në detyra të veçanta konkurente.
 - b. Balancimi - Gjatë identifikimit të detyrave që mund të ekzekutohen paralelisht, programuesit duhet të sigurohen që detyrat do të kryejnë punë të barabartë të rëndësishme së njëjtë.
 - c. Ndarja e të dhënave - të dhënat që qasen dhe përdoren nga detyrat duhet patjetër të ndahen që të ekzekutohen në bërthama të veçanta.
 - d. Varësia në të dhëna - të dhënat që qasen nga detyrat duhet patjetër të kontrollohen për ndonjë varësi në mes të dy ose më shumë detyrave.
 - e. Testimi dhe debugging - kur një program ekzekutohet paralelisht në një bërthamë, janë të mundshme disa mënyra të ekzekutimit. Testimi dhe debugging i këtyre programeve konkurente është dukshëm më i vështirë se testimi dhe debugging i aplikacioneve me vetëm një thred.
- 8) **Paralelizmet** - Në përgjithësi, janë dy lloje të paralelizmit: paralelizmi në të dhëna dhe në detyra. Paralelizmi në të dhëna fokusohet në shpërndarjen e nënbashkësive të të dhënave të njëjta në shumë bërthama dhe kryerjen e të njëjtit operacion në secilën bërthamë. Paralelizmi në detyra përfshinë shpërndarjen e detyrave (thredave) nëpër shumë bërthama. Çdo thred kryen një operacion unik.
- 9) Thredat e shfrytëzuesit përkrahen mbi kernel dhe menaxhohen pa përkrahjen e kernelit, përderisa thredat e kernelit përkrahen dhe menaxhohen direkt nga sistemi operativ.
- 10) **Modeli shumë-me-një** pasqyron shumë threda të nivelit të shfrytëzuesit në një thred të kernelit.
- 11) **Modeli një-me-një** pasqyron çdo thred të shfrytëzuesit në një thred të kernelit. Ofron më shumë konkurrencë se modeli shumë-me-një duke lejuar një thred tjetër të ekzekutohet kur një thred thirrë një thirrje sistemore bllokuese. Lejon poashtu që shumë threda të ekzekutohen paralelisht në shumë procesorë.

- 12) **Modeli shumë-me-shumë** bën bashkë shumë threda të nivelit të shfrytëzuesit në një numër më të vogël ose të barabartë të thredave të kernelit. Numri i thredave të kernelit mund të jetë specifik për aplikacionin e caktuar ose kompjuterin e caktuar.
- 13) **Modeli me dy nivele** - Një ndryshim në modelin shumë-me-shumë vazhdon të bëjë bashkë shumë threda të nivelit të shfrytëzuesit në një numër më të vogël ose të barabartë të thredave të kernelit, por lejon një thred të shfrytëzuesit të lidhet me një thred të kernelit. Ky ndryshim njihet si modeli me dy nivele.
- 14) **Libraria e thredave** i ofron programuesit një API për krijimin dhe menaxhimin e thredave. Dy mënyra kryesore të implementimit:
- Libraria plotësisht në hapësirën e shfrytëzuesit pa ndonjë përkrahje nga kerneli (POSIX Pthreads)
 - Libraria në nivel të kernelit përkrahur nga sistemi operativ (POSIX Pthreads, Windows)
- 15) **Thredat asinkron** - Përmes thredave asinkron, kur prindi të krijojë një fëmijë, prindi vazhdon ekzekutimin e tij, ashtu që prindi dhe fëmija ekzekutohen në mënyrë konkurente dhe të pavarur nga njëri-tjetri.
- 16) **Thredat sinkron** - Thredat janë të sinkronizuar kur thredi prind krijon një ose më shumë fëmijë dhe duhet patjetër të presë që të gjithë fëmijët të përfundojnë para se të vazhdojë. Këtu, thredat e krijuar nga prindi kryejnë punën në mënyrë konkurente, por prindi nuk mund të vazhdojë derisa të përfundojë tërë puna.
- 17) **Pthreads** - Mund të ofrohet në nivel të shfrytëzuesit ose në nivel të kernelit. Një API standard në POSIX (IEEE 1003.1c) për krijim dhe sinkronizim të thredave. Specifikimi, jo implementimi. API specifikon sjelljen e librarisë së thredave, implementimi varet prej zhvillimit të librarise. E zakonshme në sistemet operative UNIX (Linux & Mac OS X).
- 18) **Threat ne Windows** - Çdo thred përmban
- Një ID e thredit e cila identifikon në mënyrë unike thredit
 - Një bashkësi të regjistrave që paraqesin gjendjen në procesor
 - Një regjistër PC
 - Një stek për shfrytëzuesin, që përdoret kur thredi ekzekutohet në mënyrën e shfrytëzuesit dhe një stek për kernel, që përdoret kur thredi ekzekutohet në mënyrën e kernelit
 - Një hapësirë private për ruajtje që përdoret nga disa librari run-time dhe librari që përdoren në mënyrë dinamike (DLL)
 - Bashkësia e regjistrave, steku dhe hapësira private për ruajtje njihen si konteksti i thredit
- 19) **Strukturat kyesore të të dhënave të një thredi përfshijnë:**
- ETHREAD (blloku i thredit në ekzekutim) – përfshinë pointerin për në procesin të cilit i takon thredi dhe KTHREAD, në hapësirën e kernelit
 - KTHREAD (blloku i thredit të kernelit) – përfshinë informacion të planifikimit dhe sinkronizimit për thredit, kernel-mode stek, pointer në TEB, në hapësirën e kernelit
 - TEB (blloku i ambientit të thredit) – ID e thredit, user-mode steku, thread-local storage, në hapësirën e shfrytëzuesit
- 20) **Thredat ne Linux** - Kur të thirret fork (), një detyrë e re krijohet, krahas me një kopje të të gjithë strukturave të të dhënave të procesit prind. Një detyrë e re krijohet edhe kur thirret thirrja sistemore clone (). Megjithatë, në vend se të kopjohen të gjitha strukturat e të dhënave, detyra

e re pointon në strukturat e të dhënave të detyrës prind, varësisht prej flegave që janë bartur te clone ().

Ligj. 5

- 21) **CPU-H/D Burst Cycle** – Ekzekutimi i procesit përbëhet nga një cikël i ekzekutimit në CPU dhe pritje për H/D. CPU burst shoqëruar nga H/D burst. Shpërndarja e CPU burst është e një rëndësie kryesore.
- 22) Kur CPU-ja është duke pritur (idle), sistemi operativ duhet patjetër të përzgjedhë një nga proceset për ekzekutim në listën gati. Përzgjedhja bëhet nga planifikuesi i CPU-së, që përzgjedhë një proces prej proceseve në memorie që janë gati për ekzekutim dhe ia jep CPU-në atij procesi. Një listë e pritjes mund të implementohet si një listë FIFO, një listë me prioritet, një pemë ose thjeshtë si një listë e lidhur e parenditur.
- 23) **Planifikimi preemptive dhe nonpreemptive** - Vendimet për planifikim të CPU-së mund të merren në këto katër rrethana:
 - a. Kur një proces kalon nga gjendja running në gjendjen waiting (për shembull, si rezultat i një kërkesë për H/D ose një thirrje e wait () për përfundimin e një procesi fëmijë)
 - b. Kur një proces kalon prej gjendjes running në gjendjen ready (për shembull, kur paraqitet një ndërprerje)
 - c. Kur një proces kalon prej gjendjes waiting në gjendjen ready (për shembull, me kompletimin e H/D)
 - d. Kur një proces përfundon.Kur planifikimi bëhet në rrethanat 1 dhe 4, themi se skema e planifikimit është nonpreemptive ose cooperative. Përndryshe, është preemptive.
- 24) Përmes planifikimit nonpreemptive, kur CPU t'i jetë dhënë një procesi, procesi e mbanë CPU-në derisa të lirojë atë ose duke përfunduar ose duke kaluar në gjendjen waiting ▪ Pothuajse të gjitha sistemet operative moderne përfshirë Windows, MacOS, Linux dhe UNIX përdorin algoritme për planifikim preemptive.
- 25) Planifikimi preemptive mund të rezultojë me race conditions kur të dhënat përdoren bashkë nga disa procese. Le të marrim rastin kur dy procese përdorin të njëjtat të dhëna. Kur njëri proces është duke shkruar në të dhëna, ai proces bëhet preempted ashtu që procesi i dytë të mund të ekzekutohet. Proces i dytë pastaj tenton të lexojë të dhënat, të cilat janë në gjendje jo konsistente.
- 26) Moduli dispatcher i jep kontrollin mbi CPU procesit të përzgjedhur nga planifikuesi i CPU-së; kjo përfshinë:
 - a. Ndërrimin e kontekstit
 - b. Kalimin në mënyrën e shfrytëzuesit
 - c. Kërcimi në vendin përkatës në programin e shfrytëzuesit për të ristartuar atë program
- 27) Dispatch latency (delay) – koha që i duhet dispatcher-it për të ndalë një proces dhe startuar një tjetër për ekzekutim.
- 28) Një ndërrim vullnetar i kontekstit ndodhë kur një proces ka hequr dorë nga kontrolli mbi CPU sepse ai kërkon një resurs që aktualisht nuk është në dispozicion (bllokimi për H/D). Një ndërrim jo vullnetar i kontekstit ndodhë kur CPU-ja i është marrë një procesi, në rastin kur procesi i ka mbaruar koha ose ai është bërë preempted nga një proces me prioritet më të madh.
- 29) Kriteret për planifikim:

- a. Shfrytëzimi i CPU-së – mbajtja e CPU-së të zënë sa më shumë që është e mundur
 - b. Throughput – numri i proceseve që kompletojnë ekzekutimin e tyre brenda një kohe
 - c. Turnaround time – koha për ekzekutimin e një procesi
 - d. Waiting time – koha e pritjes të një procesi në listën gati
 - e. Response time – koha e duhur prej momentit kur është bërë kërkesa deri te përgjigjja e pare
- Është e dëshirueshme të maksimizohet përdorimi i CPU-së dhe throughput dhe të minimizohet koha e kompletimit, koha e pritjes dhe koha e reagimit (dhënies së përgjigjes).
- 30) **FCFS** - Algoritmi më i thjeshtë për planifikim të CPU-së është first-come, first-served (FCFS). Me këtë skemë, CPU-ja i jepet procesit që kërkon CPU-në i pari. Algoritmi FCFS është nonpreemptive. Koha mesatare e pritjes përmes FCFS është mjaft e gjatë.
- 31) **SJF** - Çdo procesi i shoqërohet gjatësia e CPU burst të tij të radhës. Këto gjatësi përdoren për të planifikuar procesin me kohën më të shkurtër. SJF është optimal – jep minimumin e mesatares së pritjes për një bashkësi të proceseve. Versioni preemptive quhet shortest-remaining-time-first.
- 32) **Round Robin** – algoritmi me preemption për të mundësuar kalimin në mes proceseve. Çdo proces merr një pjesë të vogël të kohës së CPU-së (pjesëza kohore q), zakonisht 10 -100 milisekonda. Pasi të kalojë kjo kohë, procesi bën preemption dhe vendoset në fund të listës ready. Nëse kemi n procese në listën gati dhe pjesëza kohore është q, atëherë secili proces merr $1 / n$ të kohës së CPU-së në copëza më së shumti deri në q pjesëza kohore në një moment. Asnjë proces nuk pret më shumë se $(n - 1) q$ njësi kohore ▪ Tajmer shkakton ndërprerje në çdo pjesëz për të planifikuar procesin e radhës.
- 33) **Sipas Prioritetit** - Një numër (i plotë) i jepet çdo procesi. CPU i jepet procesit që ka prioritetin më të madh (numri më i vogël $\equiv$ prioriteti më i madh)
- a. Preemptive
 - b. Nonpreemptive
- SJF është planifikim me prioritate ku prioriteti është vlera inverze e kohës së CPU burst të radhës
- 34) Problemi kryesor i algoritmeve të planifikimit përmes prioritetit është indefinit blocking (bllokim për një kohë të pacaktuar) ose starvation. Një proces që është gati për ekzekutim por duke pritur për CPU-në mund të konsiderohet si i bllokuar. Një zgjidhje për problemin e bllokimit për një kohë të pacaktuar është aging (vjetërimi). Vjetërimi përfshinë rritjen graduale të prioritetit të proceseve që presin në sistem për një kohë të gjatë.
- 35) Një mundësi tjetër është kombinimi i planifikimit round-robin dhe me prioritet në atë mënyrë që sistemi ekzekuton procesin me prioritetin më të lartë dhe i fut në ekzekutim proceset me prioritet të njëjtë duke përdorur planifikimin round-robin.
- 36) Gara mes thredave - Modelet shumë-me-një dhe shumë-me-shumë, libraria e thredave planifikon thredat e nivelit të shfrytëzuesit për ekzekutim në një proces të lehtë. E njohur si process-contention scope (PCS) meqë gara për planifikim është brenda procesit, dhe zakonisht bëhet përmes prioritetit të vendosur nga programuesi. Planifikimi i thredit të kernelit në një CPU është system-contention scope (SCS) – garë në mes të gjithë thredave në system
- 37) **Multiprocesorë** aplikohet në këto arkitektura të sistemit:
- a. CPU-të me shumë bërthama

- b. Bërthamat me shumë threda
- c. Sistemet NUMA
- d. Multiprocesorët heterogjen

- 38) Një qasje për planifikimin e CPU-së në sistemet multiprocesorike është që të gjitha vendimet për planifikim, përpunimi H/D dhe aktivitetet tjera të sistemit trajtohen nga një procesor i vetëm – nga master server. Procesorët tjerë ekzekutojnë vetëm kod të shfrytëzuesit. Ky multiprocesim asimetrik është i thjeshtë sepse vetëm një bërthamë qaset në strukturat e të dhënave të sistemit, duke zvogëluar nevojën për përdorim të përbashkët. Mangësi e kësaj qasjeje është që master server mund të bëhet një problem potencial ku performanca e përgjithshme mund të zvogëlohet.
- 39) **SMP** - Qasja standarde për përkrahjen e multiprocesorëve është multiprocesimi simetrik (SMP), ku çdo procesor është vet-planifikues. Planifikimi vazhdon duke bërë kontrollimin e listës gati për çdo procesor nga planifikuesi dhe përzgjedhë një thred për ekzekutim. Kjo ofron dy strategji të mundshme për organizimin e thredave që mund të planifikohen:
- a. Të gjithë thredat mund të jenë në një listë gati të përbashkët (race condition).
 - b. Çdo procesor mund të ketë listën private të thredave.
- 40) **Procesorët me shumë bërthama** - Trendi i fundit është vendosja e shumë bërthamave të procesorit në një çip. Më i shpejtë dhe harxhon më pak rrymë.
- 41) **Memory stall** - paraqitet sepse procesorët modern punojnë shumë më shpejtë se memoriet. Megjithatë, një memory stall mund të paraqitet edhe për arsye të një kesh miss. Për të zgjidhur këtë situatë, shumë dizajne të harduerit kanë implementuar bërthama me shumë threda në të cilat dy (ose më shumë) threda harduerik vendosen në çdo bërthamë. Në atë mënyrë, nëse një thred harduerik ngadalësohet përderisa është duke pritur për memorie, bërthama mund të ekzekutojë një thred tjetër.
- 42) **Chip-multithreading** (CMT) i jep çdo bërthamë disa threda harduerik. (Intel i referohet kësaj si hyperthreading.)
- 43) Janë dy mënyra për realizimin e shumë thredave në një bërthamë: coarse grained dhe fine-grained multithreading. Përmes coarse-grained multithreading, një thred ekzekutohet në një bërthamë derisa të paraqitet një ngjarje me vonesë të gjatë si rasti i qasjes në memorie. Fine-grained multithreading kalon nëpër threda në një nivel më elementar – zakonisht në kufijtë e ciklit të instruksionit.
- 44) Në një nivel sistemi operativ vendosë se cilin thred softuerik të ekzekutojë në një CPU logjik. Një nivel i dytë i planifikimit specifikon se si çdo bërthamë vendosë se cili thred harduerik të ekzekutohet. Janë disa strategji që mund të përdoren në këtë situatë. Një qasje është përdorimi i një algoritmi round-robin për të planifikuar një thred harduerik për ekzekutim në një bërthamë. Një qasje tjetër e përdorur nga Intel Itanium, një procesor që ka dy bërthama me dy threda të menaxhuar nga hardueri për bërthamë. Secili thred harduerik ka një urgjencë dinamike me vlera prej 0 deri në 7, ku 0 paraqet nivelin më të ulët të urgjencës dhe 7 nivelin më të lartë.
- 45) Balancimi i ngarkesës tenton të mbajë ngarkesën të shpërndarë përgjatë të gjithë procesorëve në një sistem SMP. Push migration – një detyrë kontrollon në mënyrë periodike ngarkesën në CPU dhe nëse gjen ngarkesë e largon detyrën prej CPU-së të ngarkuar dhe e vendosë në një

tjetër CPU ▪ Pull migration – procesorët që janë më pak të zënë e tërheqin detyrën që është në pritje në një procesor më shumë të zënë.

- 46) **Planifikimi në Windows** - Windows përdorë planifikim priority-based preemptive. Thredi me prioritetin më të madh ekzekutohet me radhë. Dispeçeri është planifikuesi. Thredi ekzekutohet derisa (1) bllokohet, (2) shpenzon kohën e tij, (3) preempted nga një thread me prioritet më të madh. Threadat real-time mund të bëjnë preempt threadat nonreal-time. Skemë me 32 nivele të prioritetit. Klasa variabile është 1-15, klasa real-time është 16-31. Prioritetin 0 ka threadi që merret me menaxhim të memories. Një listë për secilin prioritet. Nëse nuk ka ndonjë thread të gatshëm, ekzekutohet një idle thread.
- 47) Windows bën dallimin në mes procesit në foreground që është zgjedhur aktualisht në ekran dhe proceseve background që nuk janë përzgjedhur. Kur një proces kalon në foreground, Windows e rrit pjesëzën e planifikimit për një sasi të caktuar – zakonisht për 3. Kjo rritje i jep procesit foreground kohë tre herë më të gjatë para se të ndodhë një time-sharing preemption.
- 48) Në Windows 7 u paraqit user-mode scheduling (UMS), që lejon aplikacionet të krijojnë dhe menaxhojnë threadat të pavarur nga kerneli. Kështu, një aplikacion mund të krijojë dhe planifikojë threadat pa e përfshirë planifikuesin e kernelit të Windows-it.
- 49) Microsoft ofron Concurrency Runtime (ConcRT), një framework e programimit konkurrent në gjuhën C++ që është dizajnuar për paralelizëm të bazuar në detyra në procesorët me shumë bërthama. ConcRT ofron një planifikues të mënyrës së shfrytëzuesit me mundësi për ndarjen e programeve në detyra, të cilat mund të planifikohen në bërthamat që janë në dispozicion.
- 50) Windows përkrahë edhe planifikimin në sistemet me shumë procesorë duke tentuar të planifikojë një thread në bërthamën më optimale për atë thread, që përfshinë mirëmbajtjen e procesorit të preferuar ose të përdorur rishtas.

Ligj. 6

- 51) **Race Condition** - Kur disa procese u qasen dhe përdorin në mënyrë konkurente të dhënat e njëjta dhe rezultati i ekzekutimit varet nga radha e caktuar sipas të cilës ndodhë qasja, quhet race condition.
- 52) Çdo proces ka një segment të kodit, të quajtur pjesa kritike, në të cilën procesi mund të jetë duke u qasur – dhe ndryshuar – të dhënat e përbashkëta me së paku një proces tjetër. Veçoria e rëndësishme e sistemit është që, kur njëri proces është duke ekzekutuar në pjesën e tij kritike, asnjë proces tjetër nuk lejohet të ekzekutojë në pjesën e tij kritike.
- 53) Një zgjidhje për problemin e pjesës kritike duhet patjetër të plotësojë këto tre kërkesa:
- Mutual exclusion (përrjashtimi reciprok). Nëse procesi P_i është duke ekzekutuar në pjesën e tij kritike, atëherë asnjë proces tjetër nuk mund të ekzekutohet në pjesën e tij kritike.
 - Progresi. Nëse asnjë proces nuk është duke ekzekutuar në pjesën e tij kritike dhe disa procese duan të futen në pjesët e tyre kritike, atëherë vetëm ato procese që nuk janë ekzekutuar në pjesën tjetër të tyre mund të marrin pjesë në marrjen e vendimit se cili proces do të futet në pjesën e tij kritike dhe kjo përzgjedhje nuk mund të shtyhet pafundësisht.
 - Pritja e kufizuar. Ka një kufizim në numrin e herave që proceset tjera lejohen të futen në pjesët e tyre kritike pasi një proces të ketë bërë kërkesën dhe para se ajo kërkesë të plotësohet.

- 54) Përdoren dy qasje të përgjithshme për t'u marrë me pjesët kritike në sisteme operative: preemptive kernels dhe nonpreemptive kernels. Një kernel preemptive lejon që një proces të bëhet preempted përderisa është duke u ekzekutuar në mënyrën e kernelit. Një kernel nonpreemptive nuk lejon një proces që është duke u ekzekutuar në mënyrën e kernelit të bëhet preempted; një proces në mënyrën e kernelit do të ekzekutohet derisa të dalë prej mënyrës së kernelit, të bllokohet ose të dorëzojë vullnetarisht kontrollin mbi CPU.

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j)
        ;

    /* critical section */

    flag[i] = false;

    /*remainder section */
}
```

- 55) } - Zgjidhja e Petersonit

- 56) Ashtu siç e thamë në fillim të kësaj pjese, zgjidhja nga Peterson nuk është e sigurt se do të funksionojë në arkitekturat moderne të kompjuterëve për arsyen kryesore që, për të përmirësuar performancën e sistemit, procesorët dhe/ose kompajlerët mund të riradhitin operacionet e leximit dhe shkrimit që nuk kanë ndonjë varësi. Për një aplikacion me shumë threda me të dhëna të përbashkëta, riradhitja e instruksioneve mund të japë rezultate jo konsistente ose jo të pritura.
- 57) Mënyra se si një arkitekturë e kompjuterit përcakton se çfarë garantimi të memories do të ofrojë për programet aplikative njihet si modeli i memories. Në përgjithësi, një model i memories bën pjesë në njëërën prej dy kategorive:
- a. Të renditura fort, ku një ndryshim në memorie në një procesor shihet menjëherë nga të gjithë procesorët.
 - b. Të renditura dobët, ku ndryshimet në memorie në një procesor mund të mos shihen menjëherë nga procesorët tjerë.
- 58) **Memory barrier** është një instruksion që detyron çdo ndryshim në memorie të bartet te të gjithë procesorët tjerë. Kur të ekzekutohet një instruksion i pengesës së memories, sistemi sigurohet që të gjitha operacionet load dhe store kompletohen në memorie.
- 59) **Instruksionet harduerike** - Instruksione të veçanta nga hardueri që na lejojnë testand-modify të përmbatjes së një fjale, ose të bëhen swap përmbajtjet e dy fjalëve në mënyrë atomike (në mënyrë të pandërprerëshme) • Instruksioni Test-and-Set • Instruksioni Compare-and-Swap
- 60) Një vegël është një variabël atomike që ofron përditësime atomike (të pandërprerëshme) në tipet bazë të të dhënave si integers dhe Booleans
- 61) **Mutex lock** përdoret për të mbrojtur pjesët kritike dhe në atë mënyrë të parandalohen race conditions. Pra, një proces duhet patjetër të marrë lock (drynin) para se të futet në pjesën kritike; ai e liron drynin kur të dalë prej pjesës kritike. Mangësia kryesore e implementimit që jepet këtu është se kërkon busy waiting.
- 62) **Semafor S** është një variabël integjer që, përveç inicializimit (vendosjes së vlerës fillestare), qaset vetëm përmes operacioneve atomike standarde: wait () dhe signal (). Semafori numëruet

- variabël integjer që mund të marrë vlerë prej një domeni të pakufizuar • Semafor binar – variabël integjer që mund të marrë vlera në mes 0 dhe 1 • Njëjtë sikur mutex lock.
- 63) Implementimi i semaforit pa Busy waiting - Çdo semafor ka një listë të pritjes. Çdo pjesë në listën gati ka dy të dhëna: • Vlera (e tipit integjer) • Pointer te vlera e radhës në listë. Dy operacione: • block – vendosja e procesit që thirrë operacionin në listën e duhur të pritjes • wakeup – largimi i njërit prej proceseve në listën e pritjes dhe vendosja e tij në listën gati
- 64) Një tip abstrakt i të dhënave (abstract data type – ADT) – mbështjellë të dhëna me një bashkësi të funksioneve për të punuar me ato të dhëna që janë të pavarura nga çfarëdo implementimi i ADT. Një tip monitor është një ADT që përfshinë një bashkësi të operacioneve të definuara nga programuesi që ofrohen me përjashtim reciprok brenda një monitori. Tipi monitor poashtu deklaron variablat vlerat e të cilave definojnë gjendjen e një objekti (instance) të atij tipi, krahas trupit të funksioneve që punojnë mbi ato të dhëna.
- 65) Koncepti i monitorit nuk mund të garantojë që radha e paraqitur më lart do të merret në shqyrtim. Në veçanti, mund të dalin këto probleme:
 - a. Një proces mund t'i qaset një resursi pa e marrë paraprakisht të drejtën për qasje në resurs.
 - b. Një proces mund të mos e lirojë kurrë një resurs pasi t'i jetë dhënë qasja në resurs.
 - c. Një proces mund të tentojë të lirojë një resurs të cilin asnjëherë nuk e ka kërkuar.
 - d. Një proces mund të kërkojë të njëjtin resurs dy herë (duke mos e liruar paraprakisht resursin).
- 66) **Liveness** i referohet një bashkësie të veçorive që një sistem duhet patjetër të plotësojë për të siguruar që proceset bëjnë progress. Pritja për një kohë të pacaktuar është një shembull i liveness failure. Shembuj: infinite loop, busy wait etj.
- 67) **Deadlock** - çdo proces në bashkësi është duke pritur për një ngjarje që mund të shkaktohet vetëm nga një proces tjetër në bashkësi.
- 68) **Protokoll të ndërrimit të prioritetit** - të gjitha proceset që janë duke u qasur në resurset që nevojiten nga një proces me prioritet më të lartë trashëgojnë prioritetin më të lartë derisa të përfundojnë me resurset në fjalë. Kur të mbarojnë, prioritetet e tyre kthehen në vlerat e tyre të mëparshme.

Ligj. 7

- 69) **Problemi i baferit të kufizuar** - n baferë, secili mund të mbajë një element. Semaphore mutex me vlerë fillestare 1, Semaphore full me vlerë fillestare 0, Semaphore empty me vlerë fillestare n. Semafori binar mutex ofron përjashtim reciprok për qasjet në baferë dhe merr vlerë fillestare 1. Semaforët empty dhe full numërojnë numrin e baferëve të zbrazët dhe atyre të mbushur plotë. Semafori empty merr vlerë fillestare n; semafori full merr vlerën fillestare 0.
- 70) **Problemi i lexuesit-shkruesit** - Një bashkësi e të dhënave përdoret bashkë nga një grup i proceseve konkurente. Lexuesit – vetëm lexojnë në bashkësinë e të dhënave; ata nuk kryejnë ndonjë përditësim. Shkruesit – mund të lexojnë dhe shkruajnë. Problemi i parë lexues-shkrues - një situatë ku një proces shkrues nuk shkruan anjëherë. Problemi i dytë i lexuesve-shkruesve kërkon që, kur një shkrues të jetë gati, ai shkrues e kryen shkrimin sa më herët që mundet.
- 71) Drynat e lexuesit-shkruesit janë më së shumti të dobishëm në këto situata:
 - a. Në aplikacione ku është e lehtë të identifikohet se cilat procese vetëm lexojnë të dhënat e përbashkëta dhe cilat procese vetëm shkruajnë të dhënat e përbashkëta.

- b. Në aplikacione që kanë më shumë lexues se shkruar. Kjo është ashtu sepse drynat lexues-shkruar në përgjithësi kërkojnë më shumë punë shtesë për t'u krijuar se semaforët ose drynat për përjashtim reciprok.
- 72) **Problemi i filozofëve që drekojnë** - N filozofë ulen në një tavolinë të rrumbullakët me një pjatë me oriz në mes. Ata kalojnë kohën herë duke menduar e herë duke ngrënë. Ata nuk kanë ndonjë ndërveprim me fqinjët e tyre. Ndërkohë tentojnë të kapin dy shkopinjë (një në një kohë) për të ngrënë nga pjata. U duhen dy shkopinjë për të ngrënë, pastaj lirojnë të dy shkopinjët kur mbarojnë. Është një paraqitje e thjeshtë e nevojës për të dhënë disa resurse në mesin e disa proceseve në një mënyrë pa deadlock dhe ndalje (starvation).
- 73) Disa prej përmirësimeve të mundshme për problemin e deadlock janë:
- a. Lejimi që në tavolinë të ulen katër filozofë në të njëjtën kohë
 - b. Lejimi që një filozof të marrë shkopinjët e tij vetëm nëse të dy shkopinjët janë në dispozicion (për ta bërë këtë, ai duhet patjetër t'i marrë shkopinjët në një pjesë kritike).
 - c. Përdorimi i një zgjidhjeje asimetrike – pra, një filozof prej një numri tek të filozofëve merr së pari shkopin e tij të majtë dhe pastaj shkopin e tij të djathtë, përderisa një filozof prej një numri çift të filozofëve merr shkopin e tij të djathtë dhe pastaj shkopin e tij të majtë.
- 74) **Zgjidhja përmes monitorit** - Kjo zgjidhje pa deadlock imponon kufizimin që një filozof mund të marrë shkopinjët e tij vetëm nëse të dy shkopinjët janë në dispozicion. Nuk ka deadlock, por pritja për një kohë të pacaktuar është e mundur.
- 75) **Sinkronizimi brenda kernelit - Windows** - Përdorë maskimin e ndërprerjeve për të mbrojtur qasjen në resurse globale në sistemet me një procesor. Përdorë spinlocks në sistemet multiprocesorike, spinlocking-thread nuk do të bëhet asnjëherë preempt. Ofron edhe dispatcher objects user-land përmes mutexes, semaforëve, ngjarjeve dhe tajmerëve. Ngjarjet: Një ngjarje ka sjelljen e njëjtë me atë të një variable të kushtit. Tajmerët njoftojnë një ose më shumë threda kur të mbarojë koha. Dispatcher objects mund të jenë signaled-state (objekti është në dispozicion) ose non-signaled state (thredi do të bllokojë).
- 76) **Objekt i pjesës kritike** është një mutex i mënyrës së shfrytëzuesit që shpesh mund të merret dhe lirohet pa intervenimin e kernelit.
- 77) **Sinkronizimi në Linux** - Linux: Para Version 2.6 të kernelit, deaktivizon ndërprerjet për të implementuar pjesët kritike të shkurtëra. Version 2.6 dhe më vonë, plotësisht preemptive. Mutex locks janë në dispozicion në Linux për mbrojtjen e pjesëve kritike brenda kernelit. Linux ofron: Semaforët, Integerët atomic, Spinlocks, Të dy format e lexuesit-shkruarit. Në sistem me një CPU, spinlocks zëvendësohen duke aktivizuar dhe deaktivizuar kernel preemption.

Ligj. 8

- 78) **Modeli i sistemit** - Një sistem përbëhet prej një numri të kufizuar të resurseve që duhet të shpërndahen në mes një numri të thredave garues. Secili resurs i llojit R_i ka W_i kopje. Ciklet e CPU-së, fajllat dhe pajisjet H/D (si kartelat e rrjetës dhe njësitë DVD) janë shembuj të llojeve të resurseve.
- 79) Në mënyrën normale të punës, një thred mund të përdorë një resurs vetëm sipas kësaj radhe:
- a. Kërkojë - Thredi kërkon resursin. Nëse kërkesa nuk mund të plotësohet menjëherë, atëherë thredi kërkues duhet patjetër të presë derisa mund të marrë resursin.
 - b. Përdorë - Thredi mund të veprojë mbi resurs.

c. Lirojë - Thredi e liron resursin.

80) **Livelock** është një formë tjetër e liveness failure. Livelock ndodhë kur një thred në mënyrë të vazhdueshme tenton të kryejë një veprim që dështon. Livelock zakonisht ndodhë kur thredat provojnë përsëri operacionet dështuese në të njëjtën kohë.

81) **Veçoritë e Deadlock** - Deadlock mund të ndodhë nëse keto katër kushte plotësohen në të njëjtën kohë:

- Përjashtimi reciprok: vetëm një thred në një moment mund të përdorë një resurs
- Mbajtja dhe pritja: një thred që është duke mbajtur së paku një resurs është duke pritur të marrë resurse shtesë që janë duke u mbajtur nga threda tjerë
- Pa preemption: një resurs mund të lirohet vetëm në mënyrë vullnetare nga thredi që e ka atë, pasi ai thred të ketë mbaruar punë me të
- Pritje rrethore: ekziston një bashkësi $\{T_0, T_1, \dots, T_n\}$ e thredave në pritje ashtu që T_0 është duke pritur për një resurs të cilin e ka T_1 , T_1 është duke pritur për një resurs që e ka T_2 , ..., T_{n-1} është duke pritur për një resurs që e ka T_n , dhe T_n është duke pritur për një resurs që e ka T_0 .

82) **Metodat për trajtimin e deadlock** - Në përgjithësi, me problemin e deadlock mund të merremi në njëren prej tre mënyrave:

- Problemin mund të injorojmë dhe të pretendojmë që deadlock nuk paraqiten asnjëherë në sistem.
- Mund të përdorim një protokoll për të parandaluar ose shmangur deadlocks, duke u siguruar që sistemi kurrë nuk do të futet në një situatë deadlock.
- Mund të lejojmë që sistemi të futet në një situatë deadlock, gjejë atë dhe dalë prej saj.

83) Për t'u siguruar që deadlocks nuk paraqiten asnjëherë, sistemi mund të përdorë një skemë të parandalimit ose të shmangies së deadlock. **Parandalimi** i deadlock ofron një bashkësi të metodave për të siguruar që së paku njëri prej kushteve të nevojshme nuk mund të plotësohet. **Shmangia** e deadlock kërkon që sistemit operativ t'i jepen informata shtesë sa i përket asaj se cilat resurse do të kërkojë dhe përdorë një thred gjatë ekzistencës së tij.

84) **Parandalimi i Deadlock** - Pamundësimi i një prej katër kushteve të nevojshme për deadlock:

- Përjashtimi reciprok – nuk kërkohet për resurset e përbashkëta (p.sh., fajllat read-only); duhet patjetër të vlejë për resurse të përbashkëta
- Mbajtja dhe pritja – duhet patjetër të garantojë që sa herë që një thred kërkon një resurs, nuk mban resurse tjera. Të kërkohet prej thredave që të kërkojnë dhe t'u jepen të gjitha resurset e tyre para se të fillojnë ekzekutimin ose të lejohet që thredi të kërkojë resurse vetëm nëse ai nuk ka asnjë resurs. Përdorim i vogël i resurseve; mund të ndodhë pritja për një kohë të pacaktuar
- Pa preemption: Nëse një proces që është duke mbajtur disa resurse kërkon një resurs tjetër i cili nuk mund t'i jepet menjëherë atij, atëherë të gjitha resurset që është duke i mbajtur duhet t'i lirojë. Resurset e marra me preemption shtohen në listën e resurseve për të cilat është duke pritur thredi. Thredi do të ristartohet vetëm nëse mund të marrë përsëri resurset që është duke i mbajtur, si dhe resurset e reja që është duke i kërkuar
- Pritje rrethore: Të detyrohet një renditje e përgjithshme e të gjitha llojeve të resurseve dhe të kërkohet që çdo thred të kërkojë resurset sipas një radhe në rritje të numërimit

85) **Shmangia e Deadlocks** - Një metodë tjetër për shmangien e deadlocks është që të kërkohet informacion shtesë për atë se si mund të kërkohen resurset. Modeli më i thjeshtë dhe më i

dobishëm kërkon që secili thred të deklarojë numrin maksimal të resurseve të secilit lloj për të cilat mund të ketë nevojë. Algoritmi për shmangie të deadlock vazhdimisht kontrollon gjendjen e dhënies së resurseve për të siguruar që asnjëherë nuk do të ketë një kusht të pritjes rrethore. Gjendja e dhënies së resurseve definohet nga numri i resurseve të dhëna dhe në dispozicion dhe kërkesat maksimale të proceseve.

- 86) **Gjendja e Sigurt** - Sistemi është në gjendje të sigurt nëse ekziston një radhë e TË GJITHË thredave në sistem ashtu që për çdo T_i , resurset të cilat T_i ende mund t'i kërkojë mund të plotësohen nga resurset që janë aktualisht në dispozicion + resurset e të gjithë T_j , me $j < i$.
- 87) **Dalja prej deadlock** - Janë dy opsione për thyerjen e një deadlock. Njëri është që thjeshtë të abortohen një ose më shumë threda për të thyer pritjen rrethore. Tjetra është që disa resurse të bëhen preempt nga një ose më shumë threda deadlocked.
- 88) **Ndalja e proceseve dhe thredave** - Abortimi i të gjithë thredave deadlocked apo abortimi një në një kohë i proceseve derisa të eliminohet cikli i deadlock
- 89) Sipas cilës radhë duhet të bëhet përzgjedhja për abortim? 1
- Prioriteti i thredit
 - Sa gjatë ka llogaritur thredi dhe sa kohë i duhet deri në përfundim?
 - Resurset që i ka përdorë thredi
 - Rerurset që i duhen thredit që të përfundojë
 - Sa threda duhet të përfundohen?
 - A është thredi interaktiv ose batch?
- 90) **Preemption i resurseve** – Nëse kërkohet preemption për t'u marrë me deadlocks, atëherë tre çështje duhet të adresohen:
- Përzgjedhja e një viktime – Cilat resurse dhe cilat procese do të bëhen preempt? minimizimi i kostos
 - Rollback – kthimi në një gjendje të sigurt, restartimi i thredit për atë gjendje
 - Starvation – i njëjti thred mund të përzgjedhet çdo herë si viktimë, duhet patjetër të sigurojmë që një proces mund të përzgjedhet si një viktimë vetëm një numër (i vogël) të fundëm të herave. Të përfshihet numri për rollback në faktorin e kostos.

Ligj. 9

- 91) Memoria kryesore dhe regjistrat brenda çdo bërthame janë të vetmet memorie me qëllim të përgjithshëm të cilave CPU mund t'ju qaset direkt.
- 92) Qasja në regjistër bëhet brenda një cikli të CPU-së (ose më pak). Memoria kryesore mund të marrë shumë cikle, duke shkaktuar një stall. Nevojitet mbrojtje e memories për të siguruar punën në mënyrë të duhur.
- 93) Duhet të sigurohet që një process mund t'u qaset vetëm adresave brenda hapësirës së tij të adresave. Këtë mbrojtje mund të ofrojmë duke përdorë regjistra base dhe limit për të definuar hapësirën logjike të një procesi. Regjistri bazë mban adresën fizike më të vogël të lejuar; regjistri i kufirit specifikon madhësinë e rangut.
- 94) CPU duhet patjetër të kontrollojë çdo qasje në memorie të krijuar në mënyrën e shfrytëzuesit për t'u siguruar se është në mes base dhe limit për atë shfrytëzues. Instruksionet që shkruajnë në regjistrat base dhe limit janë të privileguara.
- 95) Sistemit operativ, që ekzekutohet në mënyrën e kernelit, i jepet qasje e pakufizuar në memorien e sistemit operativ dhe në memorien e shfrytëzuesit.

- 96) Adresat paraqiten në forma të ndryshme në faza të ndryshme të ciklit jetësor të programit. Adresat e kodit burimor janë zakonisht me simbole, adresat e kodit të kompajluar lidhen në adresa të zhvendosshme, pra, “14 bajt nga fillimi i këtij moduli”. Linker ose loader do të lidhë adresat e zhvendosshme në adresa absolute. Çdo lidhje pasqyron një hapësirë të adresave në një tjetër.
- 97) Lidhja me adresa e instruksioneve dhe të dhënave në adresa të memories mund të ndodhë në tre faza të ndryshme
- Gjatë kompajlimit:** Nëse vendi në memorie dihet a priori, mund të gjenerohet absolute code; duhet patjetër të rikompajlohet kodi nëse vendi i fillimit ndryshon
 - Gjatë vendosjes në memorie:** Duhet patjetër të gjenerojë relocatable code nëse vendi në memorie nuk dihet gjatë kompajlimit
 - Gjatë ekzekutimit:** Lidhja shtyhet deri në kohën e ekzekutimit, procesi mund të bartet gjatë ekzekutimit të tij prej një segmenti të memories në një tjetër
- 98) **Adresa logjike** – e gjeneruar nga CPU; referuar edhe si adresa virtuale
- 99) **Adresa fizike** – adresa që shihet nga njësia e memories
- 100) Adresat logjike dhe fizike janë të njëjta gjatë kompajlimit dhe vendosjes në memorie në skemat e lidhjes së adresave; adresat (virtuale) dhe fizike dallojnë në skemat e lidhjes në kohën e ekzekutimit. Hapësira e adresave logjike është bashkësia e të gjitha adresave logjike të gjeneruara nga programi. Hapësira e adresave fizike është bashkësia e të gjitha adresave fizike të gjeneruara nga programi
- MMU** - Pasqyrimi në kohën e ekzekutimit prej adresave virtuale në adresa fizike bëhet nga pajisja harduerike e quajtur njësia për menaxhim të memories (MMU)
 - Vlera në relocation register i shtohet (mbledhje) çdo adrese të gjeneruar nga një proces i shfrytëzuesit gjatë dërgimit në memorie. Programi i shfrytëzuesit merret me adresa logjike; asnjëherë nuk i sheh adresat fizike reale
 - Përmes vendosjes dinamike, një funksion (rutinë) nuk vendoset në memorie derisa të thirret për ekzekutim. Të gjitha funksionet mbahen në disk në një format të zhvendosshëm. Programi kryesor vendoset në DRAM dhe ekzekutohet. Kur një funksion ka nevojë të thërrasë një funksion tjetër, funksioni thirrës së pari kontrollon nëse funksioni tjetër është vendosur në memorie. Nëse nuk është vendosur, thirret linking loader për të vendosur funksionin e dëshiruar në memorie dhe përditëson tabelat e adresave të programit për të reflektuar ndryshimin. Pastaj kontrolli i bartet funksionit të ri të vendosur në memorie. Përparësia e vendosjes dinamike është që një funksion vendoset në memorie vetëm kur të nevojitet.
 - Vendosja dinamike nuk kërkon ndonjë përkrahje të veçantë prej sistemit operativ. Është përgjegjësi e shfrytëzuesve të përdorin këtë metodë për të dizajnuar programet e tyre. Sistemet operative mund të ndihmojnë programuesin, megjithatë, duke ofruar librari të funksioneve për të implementuar vendosjen dinamike.
 - Lidhja dinamike** - Lidhja statike – libraritë e sistemit dhe kodi i programit kombinohen nga loader në imazhin binar të programit. Lidhja dinamike – lidhja shtyhet deri në kohën e ekzekutimit. Një pjesë e vogël e kodit, stub, përdoret për të gjetur librarinë e duhur të procedurës që gjendet në memorie. Stub zëvendëson vetën e tij me adresën e procedurës dhe ekzekuton procedurën. Sistemi operativ kontrollon nëse procedura është në adresën e memories të procesit, nëse nuk është në hapësirën e adresave, shtohet në hapësirën e adresave. Lidhja dinamike është veçanërisht e dobishme për libraritë, sistem që njihet edhe si

shared libraries. Rasti i aplikimit për përmirësimet (patching) në libraritë e sistemit mund të nevojitet versioning.

- 6) Zakonisht memoria ndahet në dy pjesë: njëra për sistemin operativ dhe tjetra për proceset e shfrytëzuesit. Sistemin operativ mund të vendosim në vendet me adresa numra më të vegjël ose në vende me adresa numra më të mëdhenjë.
- 7) Përmes dhënies në vazhdimësi të memories, çdo proces vendoset në një pjesë të vetme të memories që është në vazhdimësi të pjesës në të cilën ndodhet procesi tjetër me radhë.
- 8) Regjistrat e zhvendosjes përdoren për të mbrojtur proceset e shfrytëzuesit nga njëri-tjetri dhe ndryshimin e kodit dhe të dhënave të sistemit operativ. Regjistri bazë mban vlerën e adresës fizike më të vogël. Regjistri i kufirit mban rangun e adresave logjike – çdo adresë logjike duhet patjetër të jetë më e vogël se adresa e kufirit. MMU pasqyron adresën logjike në mënyrë dinamike
- 9) **Dhënia e memories me pjese** - Në këtë skemë me pjesë të ndryshueshme, sistemi operativ mban një tabelë që tregon se cilat pjesë të memories janë në dispozicion dhe cilat janë të zëna. Fillimisht, krejt memoria është në dispozicion për proceset e shfrytëzuesit dhe trajtohet si një bllok i madh i memories në dispozicion, si një vrimë.
- 10) Strategjitë që përdoren zakonisht për përzgjedhjen e një vrimë të lirë nga bashkësia e vrimave të lira në dispozicion:
 - a. First-fit: Dhënia e vrimës së parë që është mjaftueshëm e madhe
 - b. Best-fit: Dhënia e vrimës më të vogël që është mjaftueshëm e madhe; duhet patjetër të kërkohej në krejt listën, përveç nëse renditja bëhet sipas madhësisë. Prodhon vrimën më të vogël të mbetur
 - c. Worst-fit: Dhënia e vrimës më të madhe; poashtu duhet patjetër të kërkohej në krejt listën
- 11) **Fragmentimi** - Fragmentimi i jashtëm: ekziston krejt hapësira e memories për të plotësuar kërkesën, por nuk është në vazhdimësi. Fragmentimi i brendshëm – memoria e dhënë mund të jetë pak më e madhe se memoria e kërkuar; ky dallim në madhësi është memorie në brendësi të pjesës, por nuk përdoret. Analizat e first fit tregojnë që nëse janë dhënë N blloqe, 0.5 N e blloqeve humbin prej fragmentimit. 1/3 mund të jetë e papërdorshme → rregulli i 50 përqindëshit.
- 12) **Kompaktësimi** - Ndërrimi i përmbajtjeve të memories për të vendosur të gjithë memorien e lirë në një bllok të vetëm. Kompaktësimi është i mundur vetëm nëse zhvendosja është dinamike dhe bëhet gjatë ekzekutimit
- 13) **Paging** - një skemë për menaxhim të memories që lejon që hapësira fizike e adresave të një procesi të jetë jo e vazhdueshme. Paging shmang fragmentimin e jashtëm dhe nevojën për kompaktësim. Paging implementohet përmes bashkëpunimit në mes sistemit operativ dhe harduerit të kompjuterit.
- 14) **Metoda baze** - Ndarja e memories fizike në blloqe të madhësisë së njëjtë të quajtura korniza. Ndarja e memories logjike në blloqe të madhësisë së njëjtë të quajtura faqe. Për të ekzekutuar një program me madhësi N faqe, duhet të gjenden N korniza të lira dhe të vendoset programi
- 15) Tabela e kornizave ka një rresht me informacion për çdo kornizë të faqes fizike, duke treguar nëse korniza është e lirë ose e zënë, nëse është e zënë, cila faqe e cilit proces (ose proceseve) e ka atë.

- 16) Adresa e gjeneruar nga CPU ndahet në: Numri i faqes (p) – që përdoret si indeks në tabelën e faqes e cila përmban adresën bazë të secilës faqe në memorie fizike dhe Offset-i i faqes (d) – kombinuar me adresën bazë për të definuar adresën fizike të memories që dërgohet në njësinë e memories
- 17) **Implementimi harduerik** i tabelës së faqes mund të bëhet në disa mënyra. Në rastin më të thjeshtë, tabela e faqes implementohet si një bashkësi e regjistrave të shpejtë të dedikuar, gjë që e bën efikase përkthimin e adresës së faqes. Megjithatë, kjo qasje zgjatë kohën e ndërrimit të kontekstit, meqë secili prej këtyre regjistrave duhet të ndërrohen gjatë ndërrimit të kontekstit. Ndryshe, tabela e faqeve ruhet në memorien kryesore dhe një regjistër bazë i tabelës së faqes (PTBR) pointon kah tabela e faqeve. Ndryshimi i tabelave të faqeve kërkon ndryshimin vetëm të këtij regjistri, duke zvogëluar dukshëm kohën e ndërrimit të kontekstit.
- 18) **TLB** - Problemi i dy qasjeve në memorie mund të zgjidhet duke përdorë një kesh harduerik të veçantë për kërkim të shpejtë të quajtur translation look-aside buffer (TLB) (e quajtur edhe si memoria asociative)
- 19) Disa TLB ruajnë address-space identifiers (ASIDs) në çdo rresht në TLB – identifikon në mënyrë unike secilin proces për të ofruar mbrojtje të hapësirës së adresave për atë proces.
- 20) **Mbrojtja e memories** implementohet duke ia shoqëruar bitin e mbrojtjes secilës kornizë për të treguar nëse lejohet qasje read-only ose read-write. “valid” tregon që faqja e shoqëruar është në hapësirën e adresave logjike të procesit dhe në atë mënyrë është një faqe e lejuar, “invalid” tregon që faqja nuk është në hapësirën e adresave logjike të procesit. Ose mund të përdoret page-table length register (PTLR)
- 21) **Faqet e përbashkëta** - Një përparësi e paging është mundësia e përdorimit të përbashkët të kodit (text editors, compilers, window systems), një gjë që është veçanërisht e rëndësishme në një ambient me shumë procese.
- 22) **Swapping** - Një proces mund të bëhet swapped përkohësisht jashtë memories kryesore në një memorie të madhe dhe pastaj të sjellet prapa në memorien kryesore për ekzekutim të vazhdueshëm. Hapësira e përgjithshme fizike e proceseve mund të tejkalojë memorien fizike.
- 23) Një operacion page out e bart një faqe prej memories fizike në memorien e madhe; veprimi i kundërt njihet si page in.
- 24) **Memoria virtuale** përfshinë ndarjen e memories logjike që shihet prej zhvilluesve nga memoria fizike. Kjo ndarje lejon një memorie virtuale shumë të madhe të ofrohet për programuesit kur vetëm një memorie e vogël fizike është në dispozicion. Lejon që hapësirat e adresave të përdoren bashkë prej disa proceseve, lejon krijim më efikas të procesit. Më shumë programe duke u ekzekutuar në mënyrë konkurente. Nevojitet më pak H/D për të vendosur ose bërë swap proceset
- 25) **Hapësira virtuale** e adresave e një procesi i referohet pamjes logjike (ose virtuale) të asaj se si ruhet në memorie një proces. Zakonisht, kjo pamje është që një proces fillon në një adresë të caktuar logjike – të themi, adresa 0 – dhe ekziston në një memorie në vazhdimësi. Mundëson hapësira të shpërndara të adresave me vrima të lëna për rritje, dynamically linked libraries, etj.
- 26) **Paging sipas nevojës** - është vendosja në memorie e një faqeje vetëm kur ajo nevojitet.
- 27) Hapat për trajtimin e Page Fault
 - a. Nëse një faqe referencohet, referencimi i parë në atë faqe do të bëjë trap te sistemi operativ - Page fault

- b. Nëse referenca ishte invalide, e përfundojmë procesin. Nëse ishte valide, por ajo faqe ende nuk është vendosur në memorie, atëherë ajo faqe vendoset në memorie.
 - c. Gjetja e një kornize të lirë
 - d. Faqja bëhet swap në kornizë përmes operacionit të planifikuar në disk
 - e. Resetohen tabelat për të treguar që faqja është në memorie, biti valid merr vlerën = v
 - f. Ristartohet instruksioni që ka shkaktuar page fault. Procesi mund t'i qaset faqes sikur ajo të kishte qenë në memorie edhe më herët.
- 28) Programet kanë tendencë të kenë referencim në afërsi, që rezulton në një performancë të arsyeshme nga paging sipas nevojës. Hardueri për përkrahje të paging sipas nevojës është i njëjtë me harduerin për paging dhe swapping.
- 29) Kornizat zero-fill-on-demand “bëhen zero” para se të jepen, kështu duke zbrazur përmbajtjet e tyre të mëparshme.
- 30) **Zëvendësimi i faqes** kompletion ndarjen në mes memories logjike dhe memories fizike – mund të ofrohet një memorie virtuale e madhe në një memorie fizike të vogël.
- 31) Zëvendësimi themelor i faqes
- a. Gjetja e vendit të faqes së dëshiruar në disk
 - b. Gjetja e një kornize të lirë: - Nëse ka një kornizë të lirë, ajo përdoret - Nëse nuk ka asnjë kornizë të lirë, përdoret një algoritëm për zëvendësim për përzgjedhjen e një kornize viktimë - Shkruarja e kornizës viktimë në disk vetëm nëse ka ndryshuar
 - c. Vendosja e faqes së dëshiruar në kornizën e re të lirë; përditësimi i tabelave të faqeve dhe kornizave
 - d. Vazhdimi i procesit duke ristartuar instruksionin që ka shkaktuar trap
- 32) **LRU** - Zëvendësimi përmes LRU-së secilës faqe ia shoqëron kohën e përdorimit të fundit të asaj faqeje. Kur një faqe duhet patjetër të zëvendësohet, LRU përzgjedhë faqen për të cilën ka kaluar koha më e gjatë prej se është përdorë herën e fundit. Dy implementime janë të përshtatshme: • Numëruesit dhe Steku.
- 33) **Thrashing** - Një proces bën thrashing nëse është duke shpenzuar më shumë kohë për paging se sa për ekzekutim. Thrashing rezulton me disa probleme të performancës.
- 34) Shkaktarët e thrashing – përdorimi i algoritmit për zëvendësim global të kornizave, rritja e tepert e nivelit të multiprogramimit etj.
- 35) Modeli i lokaliteti thotë që, kur një proces është duke u ekzekutuar, ai kalon nga lokaliteti në lokalitet. Një lokalitet është një bashkësi e faqeve që përdoren bashkë në mënyrë aktive. Lokalitetet definojnë nga struktura e programit dhe strukturat e tij të të dhënave. Modeli i lokalitetit thotë që të gjitha programet do të kenë këtë strukturë bazë të referencimit në memorie.
- 36) Linux përdorë një teknikë të zëvendësimit global. Për të menaxhuar memorien, Linux mirëmban dy lloje të listave të faqeve: një listë_aktive dhe një listë_joaktive. Lista_aktive përmban faqet që konsiderohen se janë në përdorim, përderisa lista_joaktive përmban faqet të cilat nuk janë referencuar rishtaz dhe mund të merren prapa.
- 37) Windows përdorë paging sipas nevojës me clustering. Clustering sjellë në memorie faqet që e rrethojnë faqen që ka shkaktuar page fault. Minimumi i bashkësisë së punës është minimumi i numrit të faqeve që procesi garantohej të ketë në memorie; nësa ka mjaftueshëm memorie në dispozicion, një procesi mund t'i jepen aq faqe sa maksimumi i bashkësisë së punës të tij.

- 38) Kur sasia e memories së lirë në sistem bie nën një vlerë referente, kryhet automatic working set trimming për të kthyer sasinë e memories së lire. Working set trimming largon faqet prej proceseve që kanë faqe përtej working set minimum të tyre
- 39) **Planifikimi i disqeve HDD** - Sistemi operativ është përgjegjës që hardueri të përdoret në mënyrë efikase: për njësitë e disqeve, kjo nënkupton kohë të shkurtër të qasjes dhe bandwidth të madh. Minimize seek time: Koha e kërkimit (seek time) $\approx$ distanca e kërkimit (seek distance). Bandwidth i diskut është numri i përgjithshëm i bajtëve të bartur, pjesëtuar me kohën e përgjithshme në mes kërkesës së parë për shërbim dhe kompletimit të bartjes së fundit
- 40) Kurdo që një proces ka nevojë për H/D te ose prej diskut, ai bën një thirrje sistimore te sistemi operativ. Kërkesa specifikon disa informata:
- Nëse operacioni është hyrje apo dalje
 - Informata për fajllin e hapur që tregon fajllin mbi të cilin operohet
 - Cila është adresa e memories për bartjen
 - Sasia e të dhënave për bartje
- 41) Qëllimet aktuale të planifikimit të diskut përfshijnë planifikimin e drejtë, me kohë dhe të optimizuar, si grumbullimi i shkrimeve ose leximeve që paraqiten në radhë, meqë disqet punojnë më mirë me anë të H/D sekuenciale.
- 42) Pajisjet NVM nuk kanë koka lëvizëse të diskut dhe zakonisht përdorin një teknikë të thjeshtë FCFS. Për shembull, planifikuesi NOOP në Linux përdorë një teknikë FCFS, por e ndryshon atë për të bashkuar kërkesat e afërta. H/D mund të bëhet në mënyrë sekuenciale ose të rastësishme. Qasja sekuenciale është optimale për pajisjet mekanike sikur HDD dhe shiritat magnetik sepse të dhënat që duhet të lexohen ose të shkruhen janë afër kokës për lexim/shkrim. H/D me qasje të rastit, e cila matet me input/output operations për second (IOPS), shkakton lëvizje të kokës së diskut HDD. Natyrisht, H/D me qasje të rastësishme është shumë më e shpejtë në NVM. Një HDD mund të prodhojë qindra IOPS, përderisa një SSD mund të prodhojë qindra mijëra IOPS.
- 43) **Menaxhimi i pajisjeve ruajtëse** - Low-level formatting ose physical formatting: Ndarja e diskut në sektorë të cilët kontrolluesi mund t'i lexojë ose shkruajë. Çdo sektor mund të mbajë informacion të hederit, plus të dhëna, plus error correction code (ECC).
- 44) Para se të përdorë një disk për të ruajtur fajllat, sistemi operativ ka nevojë të vendosë strukturat e tij të të dhënave në pajisje. Këtë e bën në tre hapa.
- Particionimi i diskut në një ose më shumë grupe të cilindrave, ku secili trajtohet si disk logjik
 - Hapi i dytë është krijimi i volume dhe menaxhimi. Ndonjëherë, ky hap është implicit (i nënkuptuar), si në rastin kur një sistem i fajllave vendoset direkt brenda një partitioni. Ai volume është pastaj gati për montim dhe përdoret.
 - Formatimi logjik, ose krijimi i sistemit të fajllave. Në këtë hap, sistemi operativ ruan strukturat fillestare të të dhënave të sistemit të fajllave në pajisje
- 45) Për të rritur eficiencën, shumica e sistemeve të fajllave grupojnë blloqet sëbashku në pjesë më të mëdha, shpesh të quajtura kllasterë. H/D e pajisjeve bëhet përmes blloqeve, por H/D e sistemit të fajllave bëhet përmes kllasterëve, duke siguruar në mënyrë efektive që H/D ka më shumë karakteristika të qasjes sekuenciale dhe më pak të qasjes së rastit
- 46) Disa sisteme operative u japin programeve të veçanta mundësinë të përdorin një partition si një varg të gjatë të blloqeve logjike, pa ndonjë strukturë të dhënash për sistemin e fajllave. Ky varg ndonjëherë quhet raw disk dhe H/D te ky varg njihet si raw I/O.

- 47) Root partition përmban sistemin operativ, particionet tjera mund të mbajnë sisteme tjera operative, sisteme tjera të fajllave, ose mund të jetë raw (të paformatuara). Blloku boot starton sistemin. Bootstrap ruhet në ROM, firmware. Programi bootstrap loader i ruajtur në blloqet boot të boot particionit
- 48) Sector Sparing - Mund të kërkohet nga kontrolluesi që të zëvendësojë çdo sektor bad logjikisht me një prej sektorëve rezervë.
- 49) **Menaxhimi i Swap-Space** - Përdoret për të bartur proceset në tërësi (swapping), ose faqet (paging), nga DRAM në memorien sekondare kur DRAM nuk është mjaftueshëm i madh për të gjitha proceset. Sistemet për paging thjeshtë mund të ruajnë faqet që janë nxjerrë prej memories kryesore.
- 50) Një swap space mund të jetë në dy vende: mund të jetë brenda sistemit të fajllave ose mund të jetë në një particion ndaras.

Ligj. 10

- 51) **Sistemi i fajllave** ofron mekanizmin për ruajtje on-line dhe qasjen e të dhënave dhe programeve të sistemit operativ dhe për të gjithë shfrytëzuesit e sistemit operativ. Sistemi i fajllave përbëhet prej dy pjesëve: një grup i fajllave, ku secili ruan të dhëna dhe një strukturë direktoriumi, e cila organizon dhe ofron informacion për të gjithë fajllat në sistem.
- 52) **Fajll** - është një grup i emërtuar i informacionit që kanë lidhje e që ruhet në pajisje ruajtëse sekondare. Fajllat e të dhënave mund të jenë numerike, alfabetike, alfanumerike ose binare. Fajllat mund të jenë të formës së lirë, sikur fajllat tekstual ose mund të jenë të formatuar në një formë më të ngurtë. Në përgjithësi, një fajll është një radhë e bitëve, bajtëve, rreshtave, kuptimi i të cilave përcaktohet prej krijuesit të fajllit dhe shfrytëzuesit.
- 53) Një fajll tekstual është një radhë e karaktereve të organizuara në rreshta (dhe me gjasë në faqe). Një fajll burimor është një radhë e funksioneve, ku secili prej tyre është i organizuar si deklaramente të shoqëruara me shprehje të ekzekutueshme. Një fajll i ekzekutueshëm është një radhë e kodit që loader mund të sjellë në memorie dhe ekzekutojë.
- 54) Atributet e një fajlli ndryshojnë nga një sistem operativ në një tjetër, por zakonisht përbëhet prej këtyre:
 - a. **Emri** - Emri simbolik i fajllit është informacioni i vetëm në një formë që mund të lexohet prej njeriut.
 - b. **Identifikuesi** - Kjo etiketë unike, zakonisht një numër, identifikon fajllin brenda sistemit të fajllave; është emri në një formë jo të lexueshme për fajllin.
 - c. **Lloji** - Ky informacion nevojitet për sistemet që përkrahin lloje të ndryshme të fajllave.
 - d. **Vendi** - Ky informacion është një pointer te një pajisje dhe te lokacioni i fajllit në atë pajisje.
 - e. **Madhësia** - Madhësia aktuale e fajllit (në bajt, fjalë ose blloqe)
 - f. **Mbrojtja** - Informacioni i kontrollit të qasjes përcakton se kush mund të bëjë leximin, shkrimin, ekzekutimin dhe ashtu me radhë.
 - g. **Koha, data dhe identifikuesi i shfrytëzuesit** – Ky informacion mund të mbahet për krijimin, ndryshimin e fundit dhe përdorimin e fundit. Këto të dhëna mund të jenë të dobishme për mbrojtje, siguri dhe monitorim të përdorimit.
- 55) Disa sisteme të reja të fajllave përkrahin edhe atributet e zgjerueshme të fajllit, përfshirë kodimin (encoding) e karaktereve të fajllit dhe karakteristika të sigurisë si checksum i fajllit.

56) Informacioni për të gjithë fajllat mbahet në strukturën e direktoriumit, që ndodhet në të njëjtën pajisje sikur fajllat. Zakonisht, një informatë në direktorium përbëhet prej emrit të fajllit dhe identifikuesit unik të tij. Në anën tjetër identifikuesi gjen atributet tjera të fajllit.

57) Veprimet me fajlla –

- a. **Krijimi** i një fajlli. Janë të nevojshëm dy hapa për krijimin e një fajlli. Së pari, duhet të ketë hapësirë në sistemin e fajllave për fajllin. Së dyti, një rresht me informata për fajllin e ri duhet patjetër të vendoset në një direktorium.
- b. **Hapja** (Fi) – kërkimi në strukturën e diskut për të dhënë Fi dhe bartja në memorie e përmbajtjes së të dhënës. **Mbyllja** (Fi) – bartja e përmbajtjes së të dhënës Fi në memorie në strukturën e direktoriumit të disk
- c. **Shkruarja** e një fajlli. Për të shkruar në një fajll, bëjmë një thirrje sistemore duke specifikuar të drejtën mbi fajll dhe informacionin që duhet të shkruhet në fajll. Sistemi duhet patjetër të mbajë një write pointer te vendi në fajll ku do të bëhet shkruarja e radhës nëse është sekuenciale. Write pointer duhet patjetër të përditësohet sa herë që ndodhë një shkrim.
- d. **Leximi** i një fajlli. Për të lexuar prej një fajlli, përdorim thirrjen sistemore që specifikon të drejtën mbi fajll dhe ku (në memorie) duhet të vendoset blloku i radhës në fajll. Përsëri, sistemi ka nevojë të mbajë një read pointer te vendi në fajll ku do të bëhet leximi i radhës, nëse është sekuencial. Pasi të ketë ndodhur leximi, përditësohet read pointer. Meqë një proces zakonisht është ose duke lexuar prej ose duke shkruar në një fajll, vendi aktual i veprimit mund të mbahet si një current-file-position pointer.
- e. **Ripozicionimi** brenda një fajlli. Current-file-position pointer i fajllit të hapur ripozicionohet te një vlerë e dhënë. Ripozicionimi brenda në fajll nuk duhet të përfshijë ndonjë H/D. Ky veprim në fajll njihet edhe si seek (kërkim) në fajll.
- f. **Fshirja** e një fajlli. Për fshirjen e një fajlli, kërkohet në direktorium me emrin e fajllit. Pas të jenë gjetur të dhënat përkatëse në direktorium, lirohet hapësira e fajllit, ashtu që ajo mund të përdoret nga fajlla të tjerë dhe fshijë ose shënojë si të lirë vendin në direktorium.
- g. **Shkurtimi** i një fajlli. Një shfrytëzues mund të ketë nevojë të fshijë përmbajtjet e një fajlli, por të mbajë atributet e tij. Në vend se të kërkohet që shfrytëzuesi të fshijë një fajll dhe pastaj të rikrijë atë, ky funksion lejon që të gjitha atributet të mbeten të pandryshuara – me përjashtim të gjatësisë së fajllit. Fajlli pastaj mund të resetohet në gjatësinë zero dhe hapësira e tij të lirohet.

58) Sistemi operativ mban një tabelë, të quajtur open-file table, që përmban informacion për të gjithë fajllat e hapur.

59) **Fajllat e hapur** - Disa pjesë të të dhënave nevojiten për të menaxhuar fajllat e hapur:

- a. Tabela e fajllave të hapur: mban shënime për fajllat e hapur
- b. Pointeri i fajllit: pointer për në vendin e fundit të leximit/shkrimit, për procesin që e ka hapur fajllin
- c. Numri i hapjeve të fajllit: një numërues për herët e hapjes së fajllit – për të lejuar largimin e të dhënave nga tabela e fajllave të hapur kur procesi i fundit e mbyllë atë
- d. Vendi në disk i fajllit: keshi i informacionit për qasjen në të dhëna
- e. Të drejtat e qasjes: informacion për mënyrën e qasjes në fajlla

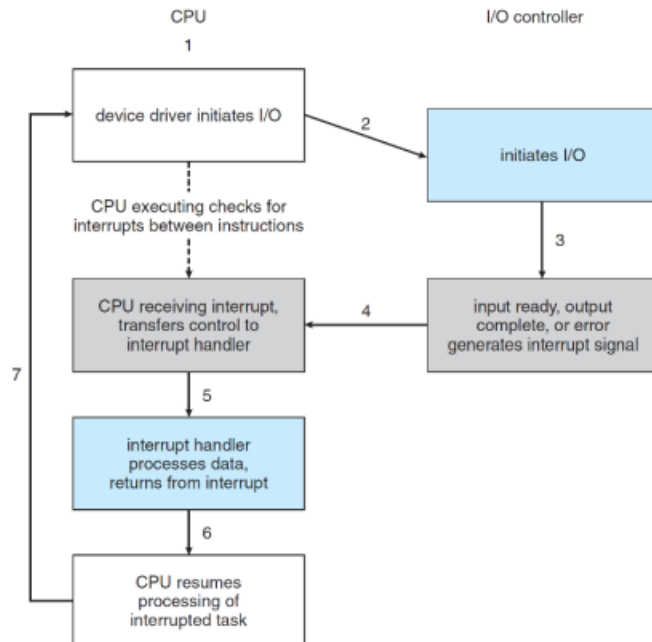
- 60) **Mbyllja e fajllit** - Ofrohet nga disa sisteme operative dhe sisteme të fajllave. Ngjashëm me drynët lexues-shkrues, ku dryni i përbashkët i ngjashëm me drynin e lexuesit – disa procese mund të marrin në mënyrë konkurente, dhe dryni ekskluziv i ngjashëm me drynin e shkruesit.
- 61) Sistemet operative mund të ofrojnë mekanizma për file-locking mandatory ose advisory. Përmes mandatory locking, pasi një proces të kërkojë një exclusive lock, sistemi operativ do të parandalojë cilindo proces tjetër që të qaset në fajllin e mbyllur. Përndryshe, nëse lock është advisory, atëherë sistemi operativ nuk do të parandalojë ndonjë proces për marrjen e të drejtës për qasje.
- 62) Emri i fajllit ndahet në dy pjesë – një emër dhe një ekstension (prapashtesë), zakonisht e ndarë me një pike.
- 63) Sistemi operativ UNIX definon të gjithë fajllat si rrjedha të thjeshta të bajtëve. Secili bajt është individualisht i adresueshëm me offset-in e tij prej fillimit (ose fundit) të fajllit. Madhësia e logical record, madhësia e bllokut fizik dhe teknika e paketimit përcaktojnë sa logical records janë në secilin bllok fizik. Paketimi mund të bëhet ose nga programi aplikativ i shfrytëzuesit ose nga sistemi operativ.
- 64) **Qasja sekuenciale** - Metoda më e thjeshtë e qasjes është qasja sekuenciale. Informacioni në fajll përpunohet në radhë, rresht për rresht. Kjo mënyrë e qasjes është më e zakonshme; për shembull, editorët dhe kompajlerët zakonisht u qasen fajllave në këtë mënyrë.
- 65) **Qasja direkte** - Këtu, një fajll përbëhet prej logical records me gjatësi fikse që lejon programet të lexojnë dhe shkruajnë records shpejtë jo në një mënyrë të caktuar. Metoda e qasjes direkte bazohet në një model disku të fajllit, meqë disqet lejojnë qasje të rastit në cilindo bllok të fajllit.
- 66) Numri i bllokut i ofruar nga shfrytëzuesi për sistemin operativ është një numër relativ i bllokut. Një numër relativ i bllokut është një indeks relativ ndaj fillimit të fajllit.
- 67) **Direktoriumi** mund të shihet si një tabelë e simboleve që përkthen emrat e fajllave në blloqet e kontrollit të fajllave të tyre.
- 68) Veprimet që kryhen në një direktorium janë:
- Kërkimi** i një fajlli. Kemi nevojë të jemi në gjendje të kërkojmë në një direktorium për të gjetur të dhënat për një fajll të caktuar. Krijimi i një fajlli. Fajlla të ri duhet të krijohen dhe të shtohen në direktorium.
 - Fshirja** e një fajlli. Kur një fajll nuk nevojitet më, duam të jemi në gjendje ta largojmë atë prej direktoriumit. Një fshirje lë një zbrazëti në strukturën e direktoriumit dhe sistemi i fajllave mund të ketë një metodë për të defragmentuar strukturën e direktoriumit.
 - Listimi i përmbajtjes** së një direktoriumi. Duhet të jemi në gjendje të listojmë fajllat në një direktorium dhe përmbajtjet e dhënave të direktoriumit për secilin fajll në listë.
 - Riemërtimi** i një fajlli. Meqë emri i një fajlli paraqet përmbajtjen e fajllit për shfrytëzuesit e tij, duhet patjetër të jemi në gjendje të ndërrojmë emrin kur përmbajtjet ose përdorimi i fajllit ndryshon. Riemërtimi i një fajlli mund poashtu të lejojë që pozita e tij në direktorium të ndryshojë.
 - Kalimi nëpër direktorium**. Mund të kemi nevojë të qasemi në çdo direktorium dhe çdo fajll brenda strukturës së direktoriumit.
- 69) Radha e direktorimeve në të cilat kërkohet për një fajll quhet search path (shtegu i kërkimit).
- 70) Mekanizmat e mbrojtjes ofrojnë qasje të kontrolluar duke kufizuar llojet e qasjeve që mund të bëhen në fajll. Qasja lejohet ose nuk lejohet varësisht prej disa faktorëve, një prej të cilëve është lloji i kërkuar i qasjes. Disa veprime të ndryshme mund të kontrollohen:

- a. **Lexo.** Lexo prej fajllit.
 - b. **Shkruaj.** Shkruaj ose rishkruaj në fajll.
 - c. **Ekzekuto.** Vendose fajllin në memorie dhe ekzekuto atë.
 - d. **Shto.** Shkruaj informacionin e ri në fund të fajllit.
 - e. **Fshij.** Fshij fajllin dhe liro hapësirën e tij për ndonjë ripërdorim të mundshëm.
 - f. **Listo.** Listo emrin dhe atributet e fajllit.
 - g. **Ndrysho atributin.** Ndryshimi i attributeve të fajllit.
- 71) Qasja më e zakonshme për problemin e mbrojtjes është që qasja të bëhet e varur prej identitetit të shfrytëzuesit. Shfrytëzues të ndryshëm mund të kenë nevojë për lloje të ndryshme për qasje në një fajll ose direktorium. Skema më e përgjithshme e implementimit të qasjes së varur nga identiteti është që çdo fajlli dhe direktoriumi t'i shoqërohet një listë e kontrollit të qasjes (access-control list) duke specifikuar emrat e shfrytëzuesve dhe llojet e qasjes të lejuara për çdo shfrytëzues.
- 72) Për të shkurtuar gjatësinë e listës të kontrollit të qasjes, shumë sisteme njohin tre klasifikime të shfrytëzuesve në lidhje me secilin fajll:
- a. Pronari. Shfrytëzuesi që ka krijuar fajllin është pronari.
 - b. Grupi. Një bashkësi e shfrytëzuesve që janë duke përdorur fajllin dhe ka nevojë për qasje të njëjtë është një grup, ose grup i punës.
 - c. Tjerë. Të gjithë shfrytëzuesit tjerë në sistem.
- 73) **Sistemi i fajllave** - Kompjuterët për qëllime të përgjithshme mund të kenë disa pajisje ruajtëse. Pajisjet mund të ndahen në particione, që përmbajnë volumes. Volumes mund të shtrihen në shumë particione. Çdo volume zakonisht formatohet me një sistem të fajllave. Numri i sistemeve të fajllave ndryshon, zakonisht janë disa në dispozicion për përgjdhje
- 74) Sikur që një fajll që duhet patjetër të hapet para se të përdoret, një sistem i fajllave duhet patjetër të montohet para se të jetë në dispozicion për proceset në sistem. Sistemit operativ i jepet emri i pajisjes dhe pika e montimit – vendin brenda strukturës së fajllit ku do të bashkangjitet sistemi i fajllave.
- 75) Particioni mund të jetë një volume që përmban një sistem të fajllave (“cooked”) ose raw – vetëm një radhë e blloqeve pa ndonjë sistem të fajllave. Boot block mund të pointojë në boot volume ose në një bashkësi të blloqeve të boot loader që përmbajnë mjaftueshëm kod që të jenë në gjendje të vendosin në DRAM kernelin prej sistemit të fajllave
- 76) **Root partition** përmban sistemin operativ, particionet tjera mund të mbajnë sisteme tjera operative, sisteme tjera të fajllave, ose mund të jenë raw.
- 77) **File Sharing** - Lejon shumë shfrytëzues/sisteme të kenë qasje në fajllat e njëjte. Lejet/mbrojtja duhet patjetër të implementohen dhe të jenë të sakta. Shumica e sistemeve ofrojnë konceptin e pronarit, anëtar i grupit. Shumica kanë një mënyrë për të aplikuar këto në mes sistemeve

Ligj. 11

- 78) Dy detyrat kryesore të një kompjuteri janë H/D dhe llogaritjet. Detyra e sistemit operativ në H/D e kompjuterit është të menaxhojë dhe kontrollojë veprimet me H/D dhe pajisje H/D.
- 79) **Drajverët e njësive** e fshehin detajet e pajisjeve, paraqesin një interfejs uniform të qasjes në pajisje për nënsistemin e H/D, ngjashëm sikur që thirrjet sistimore ofrojnë një interfejs standard në mes të aplikacionit dhe sistemit operativ.

- 80) Pajisjet H/D bëjnë pjesë në: kategoritë e përgjithshme të pajisjeve (disqe, shirita), pajisje të bartjes (lidhje të rrjetave, Bluetooth) dhe pajisje të interfejsit të shfrytëzuesit (ekran, tastierë, maus, audio in dhe out).
- 81) Pajisja komunikon me kompjuterin përmes një pike të lidhjes, ose porti – për shembull, një port serik. Nëse pajisjet përdorin bashkë një bashkësi të telave, lidhja quhet bus. Një bus, sikur busi PCI që sot përdoret në shumicën e kompjuterëve, është një bashkësi e MSc. Valdrin Haxhiu 2 telave dhe një protokoll i definuar mirë që specifikon një bashkësi të mesazheve që mund të dërgohen përmes të telave.
- 82) **PCI** - bus i zakonshëm në PC dhe serverë, **PCI Express (PCIe)**. **Expansion bus** lidhë pajisjet relativisht të ngadalshme. **Serial-attached SCSI (SAS)** interfejs i zakonshëm i diskut
- 83) Një **rrugë** përbëhet prej dy çifteve të sinjaleve, njëri çift për pranimin e të dhënave dhe tjetri për dërgim. Çdo rrugë përbëhet prej katër telave dhe secili tel përdoret si një rrjedhë full-duplex (në të dy drejtimet) e bajtëve, që bartin pako me të dhëna në një format me bajt 8 bitësh në të njëjtën kohë në të dy drejtimet.
- 84) Një **kontrollues** është një bashkësi e çipave që mund të menaxhojë një port, bus ose një pajisje.
- 85) Një **host bus adapter (HBA)** implementohet si një pllakë elektronike e veçantë – ose – që lidhet në një bus në kompjuter. Zakonisht ka një procesor, mikrokod dhe një memorie private për të mundësuar të jetë në gjendje të përpunojë mesazhet e protokollit FC.
- 86) **Hyrje/daljen e pasqyruar në memorie (Memory-mapped)** - Në këtë rast, të dhënat e pajisjes regjistrat për kontroll të pajisjes pasqyrohen në hapësirën e adresave të procesorit.
- 87) Kontrolli i pajisjeve H/D zakonisht përbëhet prej katër regjistrave:
- Regjistri **data-in** lexohet nga hosti për të marrë hyrjen.
 - Regjistri **data-out** shkruhet nga hosti për të dërguar daljen.
 - Regjistri i **gjendjes** përmban bitët që mund të lexohen nga hosti. Këta bitë tregojnë gjendje
 - Regjistri i **kontrollit** mund të shkruhet nga hosti për të filluar një komandë ose për të ndryshuar mënyrën e pajisjes.
- Regjistrat e të dhënave janë zakonisht me madhësi 1 deri në 4 bajt.
- 88) **Polling** - Për secilin bajt të H/D:
- Lexohet busy bit prej regjistrat të statusit derisa të jetë 0
 - Hosti vendosë bitin lexim ose shkrim dhe nëse biti ka vlerën për shkrim të dhënat vendosen në data-out register
 - Hosti vendosë bitin command-ready
 - Kontrolluesi vendosë busy bit, ekzekuton bartjen
 - Kontrolluesi fshin busy bit, error bit dhe command-ready bit kur të mbarojë bartja
- 89) Hapi 1 është cikli busy-wait ose polling për të pritur për H/D prej pajisjes. I arsyeshëm nëse pajisja është e shpejtë, por jo efikase nëse pajisja është e ngadalshme. Polling mund të bëhet me 3 cikle të instruksioneve: Lexohet statusi, logical-and për nxjerrjen e bitit të statusit, degëzim nëse nuk është zero
- 90) Mekanizmi harduerik që mundëson që një pajisje të njoftojë CPU-në quhet interrupt (ndërprerje). Themi që kontrolluesi i pajisjes ngritë (raises) një ndërprerje duke vendosur një sinjal në interrupt request line, CPU kapë (catches) ndërprerjen dhe dispeçon atë te interrupt handler dhe handler e përpunon ndërprerjen duke e shërbyer pajisjen.



- 91) - Funksionimi I ndërprerjes
- 92) Shumica e CPU-ve kanë dy interrupt request lines. Njëra është nonmaskable interrupt, që rezervohet për ngjarje sikur gabimet e papërmirësueshme të memories. Interrupt line e dytë është maskable: për të injoruar ose shtyrë në kohë disa ndërprerje. Mund të ç'kyqet nga CPU para ekzekutimit të radhëve kritike të instruksioneve që patjetër nuk duhet të ndërprehen. Maskable interrupt përdoret nga kontrolluesit e pajisjeve për të kërkuar shërbime.
- 93) **Interrupt vector** - adrese për të lidhur ndërprerjen me procedurën përkatëse për trajtim të saj. **Interrupt chaining** - secili element në vektorin e ndërprerjes pointon te një listë e interrupt handlers.
- 94) Mekanizimi i ndërprerjeve implementon edhe një sistem të niveleve të prioritetit të ndërprerjeve. Këto nivele mundësojnë CPU-në të shtyjë trajtimin e ndërprerjeve të prioritetit të ulët pa bërë maskimin e të gjitha ndërprerjeve dhe duke bërë të mundur që një ndërprerje e prioritetit të lartë të bëjë preempt ekzekutimin e një ndërprerjeje të nivelit të ulët.
- 95) **Exceptions** - Mekanizimi i ndërprerjeve përdoret edhe për të trajtuar një numër të ndryshëm të exceptions, sikur pjesëtimi me zero, qasja në një adresë që nuk ekziston ose të pambrojtur, ose tentimi i ekzekutimit të një instruksioni të privilegjuar prej mënyrës së shfrytëzuesit.
- 96) Sistemet shpesh ndajnë menaxhimin e ndërprerjeve në mes first-level interrupt handler (FLIH) dhe një second-level interrupt handler (SLIH). FLIH kryen ndërrimin e kontekstit, ruajtjen e gjendjes dhe vendosjen në listë të pritjes të një veprimi të trajtimit, përderisa SLIH i planifikuar ndaras kryen trajtimin e veprimit të kërkuar
- 97) **Trap (Software Interrupt)** - Thirrja sistmore ekzekutohet përmes trap për të sinjalizuar kernelin për plotësim të kërkesës
- 98) **DMA** - Kompjuterët i shmangen ngarkimit të CPU-së me PIO duke e shkarkuar një pjesë të punës me disk dhe ngarkuar te një procesor me qëllim të veçantë të quajtur direct-memory access. Tejkalon CPU-në për të bartur të dhënat direkt në mes pajisjes H/D dhe memories.
- 99) **DMA-request** dhe **DMA-acknowledge** - Kontrolluesi i pajisjes vendosë një sinjal në telin DMA-request kur një fjalë me të dhëna është në dispozicion për bartje. Ky sinjal bën që kontrolluesi

DMA të marrë busin e memories, vendosë adresën e dëshiruar në telin e adresës së memories dhe vendosë një sinjal në telin DMA-acknowledge.

- 1) Cycle stealing prej CPU-së, por shumë më efikas. Kur të përfundojë, shkakton ndërpreje për të treguar përfundimin. Versioni që është në dijeni për adresat virtuale mund të jetë edhe më efikas – DVMA (Direct virtual memory access)
- 2) Pajisjet dallojnë në shumë dimensionë: Character-stream ose block • Sequential ose random-access • Synchronous ose asynchronous (ose të dyjat) • Sharable ose dedicated • Shpejtësia e punës • read-write, read only ose write only
- 3) Interfejsi i pajisjes me blloqe përfshinë të gjitha aspektet e nevojshme për qasje në disqe dhe në pajisje tjera të orientuara në blloqe. Komandat përfshijnë read, write, seek (Random Access).
- 4) **Direct I/O** - sistemi operativ të lejojë një mënyrë të punës në një fajll e cila deaktivizon baferimin dhe locking.
- 5) Linux, Unix, Windows dhe shumë tjera përfshijnë interfejsin socket. Ndanë protokollin e rrjetës prej punës së rrjetës. Përfshinë funksionalitetin select()
- 6) Orët dhe tajmerët - Ofrojnë kohën aktuale, kohën e kaluar, tajmer. Rezolucioni normal rreth 1/60 e sekondës. Hardueri për të matur kohën e kaluar dhe për të nisur veprime quhet **programmable interval timer**.
- 7) **H/D Bllokuese** - procesi puazohet deri sa të përfundojë H/D. E lehtë për t'u përdorë dhe kuptuar, e pamjaftueshme për disa nevoja.
- 8) **Jobllokuese** - thirrja H/D kthen sa më shumë që është në dispozicion. Interfejsi i shfrytëzuesit, kopjimi i të dhënave (H/D e baferuar). Implementuar përmes multi-threading. Kthen shpejtë numrin e bajtëve të lexuar ose shkruar
- 9) **Asynchronous** - procesi ekzekutohet përderisa H/D ekzekutohet. E vështirë për t'u përdorë. Nën sistemi i H/D sinjalizon procesin kur H/D të jetë kompletuar
- 10) **H/D e vektorizuar** - Shumë baferë të ndarë mund të kenë përmbajtjet e tyre të bartura përmes një thirrjeje sistmore, duke shmangur ndërrimin e kontekstit dhe punën shtesë të thirrjes sistmore.
- 11) Planifikimi për H/D - Planifikimi i një bashkësie të kërkesave për H/D nënkupton përcaktimin e radhës së duhur për ekzekutimin e tyre. Zhvilluesit e sistemeve operative implementojnë planifikimin duke mirëmbajtur një listë të pritjes të kërkesave për secilën pajisje. Ose, sistemi operativ mund të tentojë të jetë i drejtë, ashtu që asnjë aplikacion nuk pranon shërbim të dobët ose mund të japë shërbime bazuar në prioritet për kërkesat që janë të ndjeshme në vonesa.
- 12) **Baferimi** – ruajtja e të dhënave në memorie gjatë bartjes në mes pajisjeve: Që të merret me mospërputhjen në shpejtësi të pajisjeve, që të merret me mospërputhjen në madhësinë e bartjes në mes pajisjeve, dhe të mirëmbajë “kuptimin e kopjimit”.
- 13) **Keshimi** - sistemi operativ përdorë baferë në memorien kryesore për të mbajtur të dhënat e diskut. Këta baferë përdoren edhe si një kesh, për të përmirësuar eficiencën e H/D për fajlla që përdoren bashkë nga aplikacionet ose që shkruhen dhe rilexohen shpejtë.
- 14) **Spool** është një bafer që mban daljen për një pajisje, sikur një printer, i cili nuk mund të pranojë rrjedha të përziera të të dhënave.
- 15) Rezervimi i pajisjes - ofron qasje ekskluzive në një pajisje. Thirrjet sistmore për dhënie të pajisjes dhe marrje të pajisjes
- 16) Trajtimi i gabimeve - Si një rregull i përgjithshëm, një thirrje sistmore për H/D do të kthejë një bit të informacionit rreth gjendjes së thirrjes, duke treguar sukses ose dështim. Për shembull,

një dështim i një pajisje SCSI raportohet nga protokollu SCSI në tre nivele të detajeve: një sense key që identifikon natyrën e përgjithshme të dështimit, si një dështim i harduerit ose një kërkesë jolegale; një sense code shtesë që tregon kategorinë e dështimit, sikur një parametër gabim i komandës ose një dështim i vet-testimit; dhe një sense code qualifier që jep edhe më shumë detaje, si cili parametër i komandës ishte gabim ose cili nënsistem i harduerit ka dështuar në vet-testim

- 17) **Mbrojtja e H/D** - Për të parandaluar shfrytëzuesit të kryejnë H/D ilegale, të gjitha instruksionet për H/D i definojmë si instruksione të privileguara. H/D duhet patjetër të kryhet përmes thirrjeve sistmore. Vendet e memories së pasqyruar dhe memories së porteve të H/D poashtu duhet patjetër të mbrohen
- 18) Kerneli mban informacion të gjendjes për pajisje H/D, përfshirë tabelat e fajllave të hapur, lidhjet e rrjetës, gjendjen e pajisjeve të karaktereve. Disa përdorin metodat e orientuara në objekte dhe bartjen e mesazheve për të implementuar H/D.
- 19) Menaxhimi i rrymes elektrike - Një sistem operativ do të mund të analizonte ngarkesën e tij dhe nëse do të ishte mjaftueshëm e vogël dhe hardueri është i aktivizuar, ç'kyç pjesët si CPU-ja dhe pajisjet e jashtme për H/D.
- 20) Karakteristikat kryesore që mundësojnë sistemin operativ mobil të njohur Android të menaxhojë kohëzgjatjen e baterisë: kursimi i rrymës, menaxhimi i rrymës në nivel të pjesës dhe karakteristika wakelock.
- 21) Sistemet moderne përdorin firmware advanced configuration and power interface (ACPI) që ofron kod i cili ekzekutohet si rutina (procedura) që thirren nga kerneli për njohjen e pajisjes, menaxhimin, menaxhimin e rrymës dhe gabimeve.

Ligj. 12

- 22) Problemi i mbrojtjes – duhet të sigurohet që çdo objekt qaset në mënyrë korrekte dhe vetëm nga ato procese të cilave ju lejohet ta bëjnë një gjë të tillë. Mbrojtja mund të përmirësojë besueshmërinë duke gjetur gabimet e fshehta në interfejsët në mes pjesëve të nënsistemeve.
- 23) Principi i më pak të privilegjuarit - Programeve, shfrytëzuesve dhe sistemeve duhet t'u jepen vetëm mjaftueshëm privilegje për të kryer punët e tyre. Ndarja në pjesë është veprimi i mbrojtjes të secilës pjesë të sistemit përmes përdorimit të lejimeve specifike dhe kufizimeve të qasjes.
- 24) Audit trail – incizimi i të gjitha aktiviteteve të orientuara në mbrojtje, me rëndësi për të kuptuar se çfarë dhe pse ka ndodhur
- 25) **Unaza e mbrojtjes** - Në këtë model, ekzekutimi definohet si një bashkësi e rrathëve koncentrik, ku unaza i ofron një nënbashkësi të funksionalitetit të unazës j për çdo $j < i$. Unaza më e brendshme, unaza 0, ofron krejt bashkësinë e privilegjeve.
- 26) Me shpikjen e virtualizimit, Intel definoi një unazë shtesë (-1) që të lejohet për hypervisors ose menaxhuesit e makinave virtuale, që krijojnë dhe ekzekutojnë makina virtuale.
- 27) Në procesorët ARMv7, ARM paraqiti TrustZone (TZ), që ofroi një unazë shtesë. Ky ambient më i privilegjuari ka poashtu qasje ekskluzive në veçoritë kriptografike të përkrahura nga hardueri, sikur NFC Secure Element dhe çelësi kriptografik në çip, që e bëjnë trajtimin e fjalëkalimeve dhe informacionin e ndjeshëm më të sigurt.
- 28) **Domenet e mbrojtjes** - Unazat e mbrojtjes i ndajnë funksionet në domene dhe i radhitin në mënyrë hierarkike. Një përgjithësim i unazave është përdorimi i domeneve pa hierarki. Një

sistem kompjuterik mund të trajtohet si një bashkësi e proceseve dhe objekteve. Me objekte, nënkuptojmë objektet harduerike (si CPU, segmentet e memories, printerët, disqet dhe shiritat magnetik) dhe objektet softuerike (si fajllat, programet dhe semaforët).

- 29) **Principi need-to-know** - një proces duhet të lejohej t'ju qaset vetëm atyre objekteve për të cilat ai ka autorizim. Për më tepër, në çdo kohë, një proces duhet të jetë në gjendje t'ju qaset vetëm atyre objekteve të cilat ai aktualisht i kërkon për të kompletuar punën e tij.
- 30) Principi duhet të dijë të mendohet si një politikë dhe principi i më pak të privilegjuarit si një mekanizëm për realizim të politikës.
- 31) Mundësia për të ekzekutuar një veprim në një objekt është një e drejtë e qasjes (access right). Një domen është një bashkësi e të drejtave të qasjes, ku secila është një çift i radhitur <emer i objektit, bashkësi e të drejtave>.
- 32) Domenet mund të përdorin bashkë të drejtat e qasjes. Lidhjet mund të jenë statike ose dinamike. Nëse janë dinamike, proceset mund të ndërrojnë domenin, duke ia mundësuar procesit të kalojë nga njëri domen në tjetrin
- 33) Një domen mund të realizohet në disa mënyra:
- Çdo shfrytëzues mund të jetë një domen.
 - Çdo proces mund të jetë një domen.
 - Çdo procedurë mund të jetë një domen.
- 34) **Matrica e qasjes** - Modeli i përgjithshëm i mbrojtjes mund të shihet si një matricë, e quajtur matrica e qasjes (access matrix). Rreshtat e matricës së qasjes paraqesin domenet dhe kolonat paraqesin objekte. Çdo rresht në matricë përbëhet nga një bashkësi e të drejtave të qasjes. Rreshti access (i, j) definon bashkësinë e veprimeve që një proces që është duke u ekzekutuar në domenin Di mund të thërrasë në objektin Oj.
- 35) Nëse një proces në domenin Di tenton të kryejë një veprim në objektin Oj, atëherë ai veprim duhet patjetër të jetë në matricën e qasjes. Shfrytëzuesi që krijon objektin mund të definojë kolonën e qasjes për atë objekt
- 36) Lejimi i ndryshimit të kontrolluar në përmbajtjet e rreshtave të matricës së qasjes kërkon tre veprime shtesë: copy (kopjim), owner (pronar) dhe control (kontroll).
- 37) Problemi i garantimit që asnjë informacion i ruajtur prej fillimit në një objekt mund të lëvizë jashtë ambientit të ekzekutimit të tij quhet problemi i mbajtjes.

Kode

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
```

```
int main()
{
    pid_t pid;
```

```
    /* fork a child process */
    pid = fork();
```

```
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
```

```
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
```

```
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }
```

```
    return 0;
}
```

Krijimi i një procesi tjetër përmes
thirrjes sistimore fork ()

```

#include <stdio.h>
#include <windows.h>

int main(VOID)
{
    STARTUPINFO si;
    PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
        "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
        NULL, /* don't inherit process handle */
        NULL, /* don't inherit thread handle */
        FALSE, /* disable handle inheritance */
        0, /* no creation flags */
        NULL, /* use parent's environment block */
        NULL, /* use parent's existing directory */
        &si,
        &pi))
    {
        fprintf(stderr, "Create Process Failed");
        return -1;
    }
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}

```

Krijimi i një procesi tjetër
përmes Windows API

```

message next_produced;

while (true) {
    /* produce an item in next_produced */

    send(next_produced);
}

```

Figura 3.14 Procesi prodhues përmes bartjes së mesazhit.

```

message next_consumed;

while (true) {
    receive(next_consumed);

    /* consume the item in next_consumed */
}

```

Figura 3.15 Procesi konsumues përmes bartjes së mesazhit.

```

#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    /* set the default attributes of the thread */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid, NULL);

    printf("sum = %d\n", sum);
}

/* The thread will execute in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}

```

Figura 4.11 Program në C me shumë threda përmes API Pthreads

```

#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);

```

Figura 4.12 Kodi me Pthread për join të dhjetë thredave

```

/* The thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param)
{
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 1; i <= Upper; i++)
        Sum += i;
    return 0;
}

int main(int argc, char *argv[])
{
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param;

    Param = atoi(argv[1]);
    /* create the thread */
    ThreadHandle = CreateThread(
        NULL, /* default security attributes */
        0, /* default stack size */
        Summation, /* thread function */
        &Param, /* parameter to thread function */
        0, /* default creation flags */
        &ThreadId); /* returns the thread identifier */

    /* now wait for the thread to finish */
    WaitForSingleObject(ThreadHandle, INFINITE);

    /* close the thread handle */
    CloseHandle(ThreadHandle);

    printf("sum = %d\n", Sum);
}

```

Figura 4.13 Programi me shumë threda në C përmes Windows API

```

#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5

int main(int argc, char *argv[])
{
    int i, scope;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;

    /* get the default attributes */
    pthread_attr_init(&attr);

    /* first inquire on the current scope */
    if (pthread_attr_getscope(&attr, &scope) != 0)
        fprintf(stderr, "Unable to get scheduling scope\n");
    else {
        if (scope == PTHREAD_SCOPE_PROCESS)

            printf("PTHREAD_SCOPE_PROCESS");
        else if (scope == PTHREAD_SCOPE_SYSTEM)
            printf("PTHREAD_SCOPE_SYSTEM");
        else
            fprintf(stderr, "Illegal scope value.\n");
    }

    /* set the scheduling algorithm to PCS or SCS */
    pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

    /* create the threads */
    for (i = 0; i < NUM_THREADS; i++)
        pthread_create(&tid[i], &attr, runner, NULL);

    /* now join on each thread */
    for (i = 0; i < NUM_THREADS; i++)
        pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
    /* do some work ... */

    pthread_exit(0);
}

```

Figura 5.10 API për planifikimin Pthread

```

boolean test_and_set(boolean *target) {
    boolean rv = *target;
    *target = true;

    return rv;
}

```

ura 6.5 Definimi i instruksionit atomik test_and_set ()

```

do {
    while (test_and_set(&lock))
        ; /* do nothing */

    /* critical section */

    lock = false;

    /* remainder section */
} while (true);

```

Implementimi i përjashtimit reciprok përmes test_and_set ()

```

int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;

    if (*value == expected)
        *value = new_value;

    return temp;
}

```

Figura 6.7 Definimi i instruksionit atomik compare_and_swap ()

```

while (true) {
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */

    /* critical section */

    lock = 0;

    /* remainder section */
}

```

a 6.8 Përjashtimi reciprok përmes instruksionit compare_and_swap ()

```

while (true) {
    waiting[i] = true;
    key = 1;
    while (waiting[i] && key == 1)
        key = compare_and_swap(&lock,0,1);
    waiting[i] = false;

    /* critical section */

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;

    if (j == i)
        lock = 0;
    else
        waiting[j] = false;

    /* remainder section */
}

```

•ërjashtim reciprok me pritje të kufizuar përmes compare_and_swap ()

```

monitor ResourceAllocator
{
    boolean busy;
    condition x;

    void acquire(int time) {
        if (busy)
            x.wait(time);
        busy = true;
    }

    void release() {
        busy = false;
        x.signal();
    }

    initialization_code() {
        busy = false;
    }
}

```

14 Një monitor për dhënien e një resursi të vetëm

```

while (true) {
    wait(rw_mutex);
    . . .
    /* writing is performed */
    . . .
    signal(rw_mutex);
}

```

igura 7.3 Struktura e një procesi shkruet

```

while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);
    . . .
    /* reading is performed */
    . . .
    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
}

```

igura 7.4 Struktura e një procesi lexues


```

/* thread_one runs in this function */
void *do_work_one(void *param)
{
    int done = 0;

    while (!done) {
        pthread_mutex_lock(&first_mutex);
        if (pthread_mutex_trylock(&second_mutex)) {
            /**
             * Do some work
             */
            pthread_mutex_unlock(&second_mutex);
            pthread_mutex_unlock(&first_mutex);
            done = 1;
        }
        else
            pthread_mutex_unlock(&first_mutex);
    }

    pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
    int done = 0;

    while (!done) {
        pthread_mutex_lock(&second_mutex);
        if (pthread_mutex_trylock(&first_mutex)) {
            /**
             * Do some work
             */
            pthread_mutex_unlock(&first_mutex);
            pthread_mutex_unlock(&second_mutex);
            done = 1;
        }
        else
            pthread_mutex_unlock(&second_mutex);
    }

    pthread_exit(0);
}

while (true) {
    wait(chopstick[i]);
    wait(chopstick[(i+1) % 5]);
    . . .
    /* eat for a while */
    . . .
    signal(chopstick[i]);
    signal(chopstick[(i+1) % 5]);
    . . .
    /* think for awhile */
    . . .
}

```

Figura 7.6 Struktura e filozofit *i*

Figura 8.2 Shembull i livelock